

RESEARCH

Open Access



Generating multiperspective process traces using conditional variational autoencoders

Riccardo Graziosi^{1†}, Massimiliano Ronzani^{1*†}, Andrei Buliga^{1,2}, Chiara Di Francescomarino³, Francesco Folino⁴, Chiara Ghidini², Francesca Meneghello^{1,5} and Luigi Pontieri⁴

[†]Riccardo Graziosi and Massimiliano Ronzani contributed equally to this work.

*Correspondence: mronzani@fbk.eu

¹ Fondazione Bruno Kessler, Via Sommarive, 18, 38123 Trento, TN, Italy

² Faculty of Engineering, Free University of Bozen-Bolzano, via Bruno Buozi, 1, 39100 Bolzano, BZ, Italy

³ DISI, University of Trento, Via Sommarive, 9, 38123 Trento, TN, Italy

⁴ ICAR-CNR, Via P. Bucci, 87036 Rende, CS, Italy

⁵ DIAG, Sapienza University of Rome, Via Ariosto, 25, 00185 Rome, RM, Italy

Abstract

In recent years, trace generation has emerged as a significant challenge within the Process Mining community. Deep Learning (DL) models have demonstrated accuracy in reproducing the features of the selected processes. However, current DL generative models are limited in their ability to adapt the learned distributions to generate data samples based on specific conditions or attributes. This limitation is particularly significant because the ability to control the type of generated data can be beneficial in various contexts, enabling a focus on specific behaviours, exploration of infrequent patterns, or simulation of alternative “what-if” scenarios. Moreover, many existing models primarily focus on generating traces that capture only the control-flow and temporal perspectives, neglecting crucial aspects such as resource and data perspectives, which are essential to understanding business process executions. In this work, we address these challenges by introducing a conditional model for multiperspective process data generation based on a conditional variational autoencoder (CVAE). Conditional models offer control over the generation process by tuning input conditional variables, enabling more targeted and controlled data generation. Unlike other domains, CVAE for process mining faces specific challenges due to the multiperspective nature of the data and the need to adhere to control-flow rules while ensuring data variability. Specifically, we focus on two main objectives: (i) generating complete trace executions that include control flow, temporal data, and other data attributes, with a particular focus on trace attributes and resources, as they are the most common attributes in business processes; and (ii) conditioning the trace generation on specific control flow and temporal conditions, enabling the production of traces that align to the desired execution scenarios defined by the condition context. The generated traces are then evaluated using common metrics for generative model assessment, along with additional metrics to evaluate the quality of the conditional generation.

Keywords: Process mining, Deep learning, Generative AI, Conditional models

Introduction

Process mining (PM) (van der Aalst 2016) is a research field that focuses on the analysis, monitoring, and improvement of business processes based on event logs. Within this field, generative models have emerged in recent years as crucial tools for generating new event trace samples that replicate process behavior (Tax et al. 2017;

Evermann et al. 2017; Lin et al. 2019; Mehdiyev et al. 2017; Tax et al. 2018; Nolle et al. 2018; Camargo et al. 2019; Taymouri et al. 2020). These models support a range of applications, including anomaly detection (Nolle et al. 2018; Krajsic and Franczyk 2021), predictive monitoring (Tax et al. 2017), what-if scenario analysis (Camargo et al. 2020) and conformance checking (Sani et al. 2020). An important yet underexplored aspect of trace generation is the ability to produce traces that follow different distributions from the training data, allowing exploration of various dimensions of interest within the process. These dimensions may include exploring *what-if* scenarios, expanding variants of interest (especially when significant for the process analysis but numerically infrequent), or exploring resource contingency plans.

Moreover, many state-of-the-art generative methods predominantly focus on generating process traces that capture only the *control-flow* and *temporal* perspectives. Specifically, they produce traces as sequences of activities with associated timestamps. However, many methods fail to address the *resource* and *data* perspectives, which are crucial components of business process executions.

This limitation in the current literature is particularly significant for two reasons: (i) the increasing recognition of Business Processes as complex artifacts encompassing multiple perspectives (Mannhardt 2018; Klijn et al. 2024; Adamo et al. 2017); and (ii) the capability of Predictive Process Monitoring (PPM) techniques to forecast ongoing process executions, considering the data payload of ongoing cases, which represent perspectives that go beyond the control-flow and temporal information (Di Francescomarino and Ghidini 2022).

According (Camargo et al. 2021), generative models for Process Mining can be categorized into two main families: Data-Driven Process Simulation (DDPS) and Deep Learning (DL). DDPS constructs explicit process models from data, assuming that complete information about the simulation is always available. These models are beneficial for providing insights into specific subprocesses and allow for modifying almost every aspect of the simulation. However, DDPS often relies on oversimplified assumptions, leading to unrealistic simulations and data generation. Moreover, they struggle to capture long-term dependencies. DL models are statistical models that accurately capture the correlations between features in the generated samples. Despite their accuracy, DL models are “black box” systems, making it challenging to transparently expose the underlying process model. More importantly, existing DL-based generative models are rigid, limiting the generation of distributions different from the training data. This constraint significantly inhibits the exploration of specific scenarios or dimensions of interest. Hybrid models, which integrate the accuracy of DL techniques with the transparency of explicit process models, have been recently introduced (Camargo et al. 2022; Meneghello et al. 2023). While they may have potential to generate specific data for dimensions of interest, an assessment of this capability is still missing.

Conditional generative models (Sohn et al. 2015) have been proven effective in various domains to mitigate the rigidity observed in DL generative models. These models generate outputs influenced by certain input variables. This offers a means to guide the generation process based on desiderata for the expected output.

In this work, we introduce a conditional variational autoencoder (CVAE) model for generating traces based on LSTM neural networks. Compared to other domains, developing a CVAE for process mining presents two main challenges:

- Process execution data have an inherently multi-perspective nature. Traces consist of sequence of temporal features with both categorical (activities) and numerical (timestamps) characteristics. This complexity is further increased by the inclusion of payloads. Moreover, all these components are strongly interconnected. Therefore, due to their diverse nature, each component of the generated samples requires a dedicated module in the model architecture, which must still produce a coherent output.
- While it is desirable for the generative model to produce data with some variability, event sequences must adhere to causal constraints and diverse contextual factors. Thus, the variability must remain consistent with process constraints to ensure meaningful results.

A further contribution of our work is the proposal of a novel evaluation methodology for assessing process generation quality in a conditional context. Alongside common metrics for evaluating generative process models, such as the accuracy of the control flow and generated timestamps and resources (Chapela-Campa et al. 2023), we introduce a new evaluation procedure to measure the impact of the conditioning variable and measure the actual conditioning rate. Moreover, unlike many simulation scenarios that aim to closely replicate historical data, generative models in a conditional context should introduce variability in the generated output while remaining within process constraints. Thus, our evaluation framework includes metrics to assess the variability and conformance of the generated traces with the process rules.

By integrating conditional models into process data generation, this work aims to enhance the flexibility and control of DL generative models in Process Mining. Evaluation on four different examples based on three real-world event logs shows promising results. Specifically, the generated traces are accurate, exhibit good variability, comply with process constraints, and demonstrate that the conditional generative model effectively controls the types of traces produced.

This paper is an extended and revised version of our previous conference paper (Graziosi et al. 2024), in which we address the multiperspectiveness of the generation and its assessment. The advancements in this version focus on the following key aspects:

- *Enhanced model capabilities:* We present a new version of our conditional generative model, that incorporates the generation of additional attributes beyond timestamps. Specifically we focus on generating resources, as they are the most commonly found dynamic features in business process data. This demonstrates the model's ability to generate both numerical (timestamps) and categorical (resources) dynamic features.
- *Improved evaluation:* We enhance the evaluation of our method in two ways: (i) by expanding the comparison with other state-of-the-art generative models in PM based on a Generative Adversarial Network (GAN) approach; and (ii) by enlarging the set of evaluation metrics used to assess the generation quality, with a particular emphasis on the resource and data perspective.

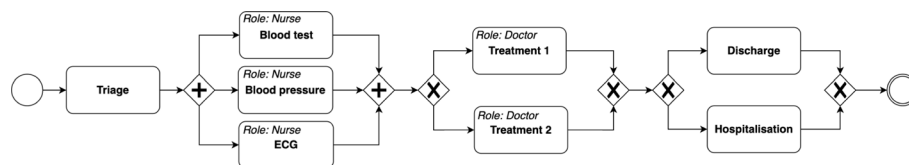
The paper is structured as follows: “[Motivating example](#)” section provides a motivating example for conditional generation and introduces key concepts through an example-driven and intuitive approach; “[Related work](#)” section reviews related works; “[Background](#)” section introduces the core technical background; “[Approach](#)” section presents our proposed solution for trace generation based on the Conditional Variational Autoencoder; “[Evaluation](#)” section defines the evaluation settings; “[Results](#)” section presents and discusses the evaluation results; “[Conclusions and future directions](#)” section concludes the paper.

Motivating example

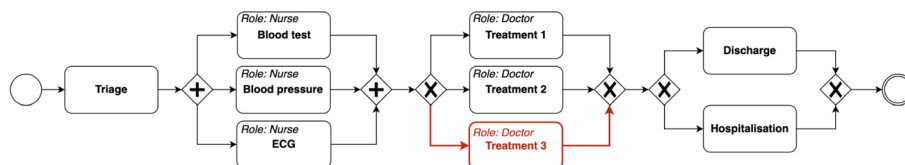
In this section, we provide a motivating example that showcases the advantages of conditional generative models. The example also introduces, in a concrete manner, various concepts discussed in this paper, such as the nature of the conditional variables, and the notions of resource and role.

Process model, attributes, resources and roles Consider a generic healthcare process for the treatment of a disease. Figure 1a presents the process model, drawn using the BPMN notation (Object Management Group BPMN Technical Committee 2011), while Table 1a provides two executions of this process (that is, two *process traces*) identified with case IDs 1 and 2, respectively. According to the process model a patient must undergo three examinations (Blood test, Blood Pressure, and ECG) and then they are assigned one of two possible treatments (Treatment 1 or Treatment 2). If the treatment is effective, the patient is discharged; otherwise, they need to be hospitalized. The two traces in Table 1a represent the care pathway of two patients and include *trace attributes* that capture demographic information, specifically age and sex of the patients along with specific information on *roles and resources* (see below).

With an abuse of the BPMN notation, Fig. 1a also contains information on the *roles* that execute certain activities. In particular, this process involves two roles: nurses and doctors. Nurses execute the three examinations while doctors perform the two



(a) Before the concept drift.



(b) After the concept drift

Fig. 1 Process models of an healthcare process for treating a disease, affected by a concept drift

Table 1 Event log with concept drift

Case_ID	age	sex	activity	timestamp	role	resource	c
(a) Before concept drift: Event log entries for two traces prior to concept drift.							
1	45	M	Triage	28/02 08:00	-	-	False
1	45	M	Blood Test	28/02 08:15	Nurse	Nati	False
1	45	M	Blood Pressure	28/02 08:30	Nurse	Ugo	False
1	45	M	ECG	28/02 09:30	Nurse	Nati	False
1	45	M	Treatment 1	28/02 11:00	Doctor	Mario	False
1	45	M	Hospitalization	28/02 16:00	-	-	False
2	60	F	Triage	28/02 09:00	-	-	False
2	60	F	Blood Test	28/02 09:15	Nurse	Ugo	False
2	60	F	ECG	28/02 09:45	Nurse	Nati	False
2	60	F	Blood Pressure	28/02 10:45	Nurse	Carlo	False
2	60	F	Treatment 2	28/02 11:00	Doctor	Anna	False
2	60	F	Discharge	28/02 17:00	-	-	False
(b) After concept drift: Event log entries for two traces following concept drift.							
3	70	M	Triage	01/03 08:30	-	-	True
3	70	M	Blood Pressure	01/03 08:45	Nurse	Carlo	True
3	70	M	Blood Test	01/03 09:00	Nurse	Ugo	True
3	70	M	ECG	01/03 09:30	Nurse	Nati	True
3	70	M	Treatment 1	01/03 11:00	Doctor	Mario	True
3	70	M	Hospitalization	01/03 16:00	-	-	True
4	55	F	Triage	01/03 09:15	-	-	True
4	55	F	Blood Test	01/03 09:30	Nurse	Ugo	True
4	55	F	Blood Pressure	01/03 10:00	Nurse	Carlo	True
4	55	F	ECG	01/03 10:30	Nurse	Nati	True
4	55	F	Treatment 3	01/03 11:15	Doctor	Anna	True
4	55	F	Discharge	01/03 14:00	-	-	True

treatments. We assume that, in the specific hospital of this example, we have three *resources* assigned to the role nurse (Carlo, Nati, and Ugo) and two *resources* assigned to the role doctor (Anna and Mario). The two traces in Table 1a provide information on who executed the specific examinations and treatments for patients 1 and 2, respectively. For example the second row indicates that the activity Blood Test is executed by nurse Nati for patient 1, while the eighth row says that the same test was executed by nurse Ugo for patient 2.

Let us assume that a large historical dataset of process executions is available for this specific process. This dataset, also called event log, is used by analysts and hospital management to investigate various aspects of the current process using process mining techniques. Some of the key areas of analysis include the adoption and effectiveness of each treatment, durations and waiting times for different process paths (variants), and the forecasting of hospitalization numbers based on the demographics of incoming patients and treatment adoption.

Concept drift scenario Let us now assume that a new treatment (Treatment 3) is made available in the hospital (Fig. 1b) at some point in time such as the 1st of March. This modification generates a *sudden concept drift* (Bose et al. 2014) in the process model which can be also seen in the type of execution traces that become possible from the

1st of March. As an example, Table 1b contains two traces of executions performed on March, 1st, and one of them contains the new treatment (Treatment 3, executed by doctor Anna).

Concept drifts can have a profound impact on process executions (see Bose et al. (2014) for details). In our example, the new treatment will have a specific appropriateness and effectiveness in curing the disease, altering the overall adoption rate of all treatments administered by doctors. In turn, this may (and, hopefully, should) affect the balance between discharges and hospitalizations. Additionally, the duration of Treatment 3 may differ significantly from the previous treatments, leading to changes in the trace duration distribution and therefore producing an impact on the use of resources. These are just some of the potential consequences of this process drift, as the new treatment may also introduce more subtle changes in the process dynamics.

As a consequence of the process drift, the large historical dataset of process executions becomes rapidly obsolete, and the analysts and hospital management may have to rely mainly on the new traces generated after the introduction of the new treatment. However, in the weeks (or even months) after the occurrence of a concept drift, only a limited number of traces can be available. This can have serious consequences on the effective application of statistical and process mining methods. Ideally, the analysts and hospital management would like to have a reliable big dataset of the new process executions to continue to perform their analysis without having to wait for weeks or months.

Conditional generative models and conditional variable Conditional generative models can be an effective way to generate traces that include the changes introduced by the concept drift (that is, traces for the new process), while still leveraging and reusing statistical patterns from the whole set of historical traces. In our example we can use this method to generate traces that include one of the three available treatments, while reusing statistical patterns from the traces that were collected until the 28th of February. To achieve this, we define a simple binary *conditional variable* c that is *True* for traces occurring after the concept drift—that is from the 1st of March—and *False* for traces occurring before the concept drift—that is until the 28th of February. By training the model on the entire event log (that is, including legacy traces from before the concept drift) and later using it to generate only *True* traces, we can produce realistic synthetic data for the new treatment scenarios, even when real data is initially sparse. This synthetic dataset can then be used to support robust analyses and ensure the continued effectiveness of process mining techniques despite the concept drift.

A good conditional generative model should be able to distinguish between trace characteristics that are strongly correlated with the conditional value (e.g., *True* for post-concept drift) and those that are independent of it. For those characteristics that are independent of the conditional variable, the model can leverage the richer statistical data from the *False* traces. This allows the model to generate more realistic synthetic data by extending beyond the limitations of the *True* traces' statistics, providing a broader and more generalized representation of the process.

As a final remark we note that the example introduced here is just one of the different scenarios that could benefit of methods for the generation of synthetic data in process mining. Real-world event logs are often scarce, incomplete, or inaccessible due to confidentiality constraints, making it challenging to apply process mining techniques effectively. Synthetic data offers a solution by enabling the creation of diverse event logs based on original datasets, allowing researchers to test and refine their models in a controlled setting. It also enables the generation of diverse scenarios, including rare or extreme cases, which may not be well represented in available real-world data. In this context, conditional deep learning models are particularly well-suited for generating realistic synthetic event logs with specific characteristics or features, based on the specified condition. These models allow for the exploration of targeted scenarios or dynamic changes in the process, such as concept drift.

Related work

Predictive Process Monitoring (Di Francescomarino and Ghidini 2022) is a recognized subfield of Process Mining (van der Aalst 2016) that focuses on forecasting the future behavior of ongoing process executions based on partial traces, i.e., the sequences of events observed up to a given point in time. Several prediction tasks have been explored in this area, including next-event prediction, suffix prediction, and entire event log generation. Among these, suffix prediction is particularly challenging and also serves as a basis for many trace generation methods (Tax et al. 2017; Evermann et al. 2017; Lin et al. 2019; Camargo et al. 2019; Jalayer et al. 2020). In recent years, Deep Neural Networks (DNNs) have been widely adopted for these tasks, with a strong emphasis on Recurrent Neural Networks (RNNs)-in particular, Long Short-Term Memory (LSTM) architectures. Accordingly, this section categorizes existing works primarily by their underlying deep learning architectures, discussing the specific tasks each approach addresses within its respective model class.

LSTM-based approaches

Several works have explored LSTM-based architectures to address the suffix prediction task, as they can capture long-term dependencies in event sequences. For example, Tax et al. (2017) proposed a dual-task LSTM model to predict both the type of the next event and its timestamp. This approach employs one-hot encoding for event types and supplements them with temporal features. By iteratively predicting subsequent events, the model can also forecast the entire remaining sequence of events and the cycle time of a case. However, the reliance on one-hot encoding introduces challenges for processes with a large number of distinct activities. To address encoding limitations, Evermann et al. (2017) introduced an alternative approach that uses embeddings for categorical attributes such as event types and resources. Although this method reduces input dimensionality, it cannot predict numerical attributes like timestamps. An LSTM-based architecture was proposed in Lin et al. (2019) to predict both the next event and its categorical attributes, including control-flow information and case data. The architecture incorporates encoders, modulators, and decoders, with the modulator providing attribute-specific relevance weights for future predictions. Despite its innovations, this model does not support predicting numerical attributes such as timestamps.

Other methods have employed different strategies. For example, Camargo et al. (2019) proposed multiple LSTM-based models capable of predicting event types, timestamps, and roles, combining features from earlier works (Tax et al. 2017; Evermann et al. 2017). In Camargo et al. (2021), the *DeepGenerator* method proposed in Camargo et al. (2019) is extended to train generative DL models for the task of generating complete event logs. The model incorporates a random sampling mechanism to enhance variability in generated traces. However, because of the sampling strategy, this approach may occasionally produce traces inconsistent with the global process distribution. Innovative architectures have been designed to address the challenge of suffix prediction in process mining. For example, Jalayer et al. (2020) proposed an LSTM-based encoder-decoder model augmented with attention mechanisms to predict suffixes iteratively. The model selects the most likely activity based on the current context at each step, progressively constructing the predicted sequence.

GAN-based approaches

Generative Adversarial Networks (GANs) have been increasingly explored in predictive process monitoring for their ability to model complex data distributions through adversarial training. For instance, Taymouri et al. (2020) introduced an encoder-decoder architecture based on LSTMs framed within a GAN setup. In this approach, the encoder-decoder acts as a generator that produces suffixes of ongoing process traces. A discriminator evaluates these suffixes, also implemented as an LSTM, distinguishing between synthetically generated suffixes and those derived from real process logs. This adversarial training mechanism promotes the generation of suffixes that closely resemble real-world data. Building on this foundation, Agarwal et al. (2022) extended the GAN-LSTM framework in Taymouri et al. (2020) to incorporate Reinforcement Learning. This enhancement ensures that the predicted suffixes are not only realistic but also compliant with the structural constraints of process models. Moreover, the model optimizes for specific key performance indicators (KPIs), providing predictions aligned with business objectives. More recently, Li et al. (2024) introduced *ProcessGAN*, a conditional GAN framework specifically designed to generate realistic process logs. This framework uses a transformer-based generator, conditioned on case duration, to synthesize activity and timestamp sequences while adhering to temporal constraints. It employs multi-head self-attention to capture long-term dependencies in activities and temporal features. Timestamps are generated as scaled differentials, ensuring their sum matches the input duration for temporal plausibility. The discriminator complements the generator by incorporating a time-aware mechanism to evaluate the quality of the synthetic data. A transformer-based context generator enhances the framework by linking contextual data to synthetic traces. This multi-task model learns from real data to predict and enrich logs with likely contextual features.

Despite the advancements in GAN-based models, comparative studies highlight their limitations. For instance, Camargo et al. (2021) compared (Taymouri et al. 2020) with other architectures, such as the LSTM-based model (Camargo et al. 2019) and a GRU-based variant of the same method, finding that the original LSTM-based method often achieved superior predictive performance. Similarly, Ketykó et al. (2022) evaluated suffix prediction methods across diverse architectures, including autoencoders, transformers, and GANs,

finding that no single approach consistently outperforms the others across different datasets. The results suggest that no single architecture consistently outperforms others across different datasets. These findings indicate that the choice of architecture may depend on the specific process characteristics, the nature of the attributes to be predicted, and practical considerations such as model interpretability and training stability. In particular, while GANs can generate realistic samples, they are prone to challenges such as mode collapse, where the generator focuses on limited regions of the data distribution (Sharma et al. 2024; Tomar and Gupta 2023).

VAE-based approaches

Variational Autoencoders (VAEs) offer a promising alternative to GANs, particularly for applications requiring robust modeling of multimodal distributions (Sharma et al. 2024), such as process logs. Unlike GANs, VAEs provide a probabilistic framework that, through latent space, inherently supports variability in generated outputs while avoiding adversarial training pitfalls such as mode collapse (Sharma et al. 2024). Although their use in predictive process monitoring is limited, some applications demonstrate their potential. For instance, Krajsic and Franczyk (2021) applied VAE-based models to anomaly detection, leveraging their ability to model the normal behavior of processes.

However, to the best of our knowledge, the application of VAEs for generative purposes in predictive monitoring remains largely unexplored. An initial exploration of this approach is presented in Graziosi et al. (2024), an abridged version of the work detailed in this manuscript. Our study represents the first comprehensive effort to leverage VAEs for the conditional generation of activity sequences, timestamp sequences, and data payloads, offering a compelling alternative to GAN-based methods such as (Li et al. 2024).

Background

In this section we introduce the main concepts useful to understand the remainder of the paper.

Event log

An event log $\mathcal{L}$ records the executions of a business process in terms of execution *traces*. A trace x consists of a sequence of ordered events $x = \langle e_1, e_2, \dots, e_n \rangle$, where each event e_i represents the execution of an activity. Events are therefore characterized by multiple attributes (*data attributes*): primarily, an event refers to an activity label and a timestamp indicating when the activity was executed. It may also include information about the resource that executes or initiates the activity and other data attributes. Some attributes are static and consistent throughout the trace execution, known as *trace attributes*, while others are associated with specific events and are known as *event attributes*. Examples of event attributes include the timestamp, which has numerical values, and the resource, which has categorical values. Data associated with events and traces in event logs are also called *data payloads*. We can represent a trace x as:

$$x = \langle (a_1, T_1, r_1, \mathbf{d}_1), \dots, (a_n, T_n, r_n, \mathbf{d}_n) \rangle \quad (1)$$

where a_i is the i -th activity executed in the trace, T_i is its timestamp, r_i the resource executing the activity, and $\mathbf{d}_i$ is a vector containing its data payload, which contain other trace and events attributes.

A *variant* is defined as the equivalent class of all the traces with the same control flow, that is the same sequence of activities. The variant associated with the trace x in (1) corresponds to the sequence $\langle a_1, a_2, \dots, a_n \rangle$. Then the set of variants $\mathcal{V}_{\mathcal{L}}$ of the event log $\mathcal{L}$ is defined as the set of all the different sequences of activities appearing in the log.

Resources and roles

The term *resource* typically refers to the individual entity executing an activity. Resources are usually exclusive, meaning that a resource can only perform one activity at a time.

In some event logs, information about the *role* of a resource may also be available. A role represents an organizational structure or classification in which resources are grouped. Resources within the same role typically have similar responsibilities and are capable of performing the same set of activities.

The concept of a role is therefore more abstract and coarser than that of individual resources. Even when information about roles is not directly available in the event log, techniques exist to derive roles from data, such as those described in Song and van der Aalst (2008). These techniques analyze the activity profiles of resources—records of how frequently each resource performs different activities—and group together resources with similar profiles.

Conditional variational autoencoders (CVAEs)

Autoencoders are neural network architectures used for unsupervised learning tasks (Hinton and Salakhutdinov 2006). They consist of an encoder, E_ϕ , and a decoder, D_θ , representing non-linear transformations parametric to θ and ϕ , respectively. The encoder maps the input data x into a latent representation $\mathbf{z} = E_\phi(x)$, while the decoder output $\hat{x} = D_\theta(\mathbf{z})$ should reconstruct the original input from the latent representation. Mathematically, an autoencoder aims to minimize the reconstruction error between the input and the output:

$$\mathcal{L}_{\text{AE}}(x, \theta, \phi) = J_{\text{rec}}(x, \hat{x}). \quad (2)$$

where J_{rec} denotes some kind of distance/error function over the data space (e.g., $\|x - \hat{x}\|^2$ in the case $x, \hat{x} \in \mathbb{R}^N$).

Variational autoencoders (VAEs) (Kingma and Welling 2014) lift this basic autoencoder architecture to the level of a generative model where x and $\mathbf{z}$ are interpreted as observed and latent random variables, respectively, such that the joint distribution of x and $\mathbf{z}$ is factored as $p_\theta(x|\mathbf{z}) \cdot p(\mathbf{z})$, where $p_\theta(x|\mathbf{z})$ is a distribution to be learned, and the latent prior $p(\mathbf{z})$ is typically set to a standard multivariate Gaussian distribution (i.e., $p(\mathbf{z}) = \mathcal{N}(\vec{0}, \mathbf{I})$, where $\mathbf{I}$ is an identity matrix)¹.

VAEs are trained to minimize the (expectation over the real data distribution of the) following negative *Evidence Lower Bound* (ELBO) $\mathcal{L}_{\text{VAE}}$, consisting of a reconstruction

¹ This allows for easily generating any $\hat{x}$ using a two-phase scheme: first $\mathbf{z}$ is sampled from the latent prior $p(\mathbf{z})$, and then $\hat{x}$ is sampled from $p_\theta(x|\mathbf{z})$.

loss term (echoing that in Eq. (2)) plus a regularization term, which is computed as the Kullback-Leibler (KL) divergence between a learned latent distribution $q_\phi(\mathbf{z}|x)$ and the latent prior $p(\mathbf{z})$, with a factor β controlling the strength of the regularization:

$$\mathcal{L}_{\text{VAE}}(x, \theta, \phi) = J_{\text{VAE}}(x, \theta, \phi) + \beta \cdot \text{KL}(q_\phi(\mathbf{z}|x) \parallel p(\mathbf{z})) \quad (3)$$

where ϕ and θ denote the parameters of the encoder and decoder sub-nets, now modeling the learned distributions $q_\phi(\mathbf{z}|x)$ and $p_\theta(x|\mathbf{z})$, respectively, while the reconstruction term is $J_{\text{VAE}}(x, \theta, \phi) = -\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|x)} \ln p_\theta(x|\mathbf{z})^2$.

Conditional variational autoencoders (CVAEs) incorporate conditional information into both the encoder and the decoder. In CVAEs, both the encoder and the decoder take the input data x and conditioning variables c as inputs (Sohn et al. 2015). The conditional variable c represents the specific condition or attribute that guides the generation process, enabling the model to produce data samples with desired characteristics. In practice, c is usually a single value or vector encoding a numerical or categorical trace-level variable. The encoder maps x and c to the parameters of a (variational) posterior distribution $q_\phi(\mathbf{z}|x, c)$ of the latent variables $\mathbf{z}$, while the decoder models a distribution $p_\theta(x|\mathbf{z}, c)$ for generating/reconstructing the input data conditioned on both the latent representation and the conditioning variables. Optimal parameters for both the encoder and decoder are learned by minimizing (the expectation over the real data of) the following negative ELBO objective:

$$\mathcal{L}_{\text{CVAE}}(x, c, \theta, \phi) = J_{\text{CVAE}} + \beta \cdot \text{KL}(q_\phi(\mathbf{z} | x, c) \parallel p(\mathbf{z} | c)) \quad (4)$$

where $q_\phi(\mathbf{z}|x, c)$ is the (learned) conditional posterior distribution of the latent variables (given both the observed data and conditioning variables), $J_{\text{CVAE}}(x, c, \theta, \phi) = -\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|x, c)} \ln p_\theta(x|\mathbf{z}, c)$, and $p(\mathbf{z}|c)$ is a prior distribution (conditioned on c) for the latent variables $\mathbf{z}$. Usually, for any given x and c , $q_\phi(\mathbf{z}|x, c)$ is assumed to be a multivariate Gaussian distribution with a diagonal covariance matrix.

Approach

In this section we present a conditional variational autoencoder (CVAE) architecture for the generation of traces. The generated traces should be representative of the original data distribution, while exhibiting a good degree of variability. Moreover, the generated traces could pertain to variants that were not previously observed, but that nonetheless comply with the process constraints of the original data.

This work is an extended version of our previous conference paper (Graziosi et al. 2024). The model presented therein allowed the generation of traces with activities a_i , timestamps T_i , and static numerical $\mathbf{d}_{\text{num}}$ and categorical $\mathbf{d}_{\text{cat}}$ traces attributes. The principal difference in this extension, is the generation of the *resource* r_i associated to each event. The trace (1) can then be rewritten as

² If setting $\beta = 1$ as in Kingma and Welling (2014) (factor β was actually introduced in a later work), the opposite of $\mathcal{L}_{\text{VAE}}(x, \theta, \phi)$ is indeed a lower bound for the (marginalized) log-likelihood $p(x)$ (a.k.a. *evidence*), since $-\mathcal{L}_{\text{VAE}}(x, \theta, \phi) = \ln p(x) - \text{KL}(q_\phi(\mathbf{z}|x) \parallel p(\mathbf{z}|x))$. Thus, finding the values of θ and ϕ that maximize the sum of $\mathcal{L}_{\text{VAE}}$ over the training data—i.e., optimally fitting the “encoder” distribution $q_\phi(\mathbf{z}|x)$ and the “decoder” distribution $p_\theta(x|\mathbf{z})$ —is a viable alternative to maximizing $p(x)$ directly, which would entail integrating over a continuous latent space. The learned distribution $q_\phi(\mathbf{z}|x)$ hence plays as an approximated (variational) distribution in the perspective of variational bayesian inference (Kingma and Welling 2014).

$$x = (\mathbf{d}_{num}, \mathbf{d}_{cat}) \otimes \langle (a_1, T_1, r_1), \dots, (a_n, T_n, r_n) \rangle$$

where the trace attributes $(\mathbf{d}_{num}, \mathbf{d}_{cat})$, which remain constant across the sequence, are explicitly factored out of the sequence.

The new architecture closely resembles the one presented in Graziosi et al. (2024) and it is displayed in Fig. 2, where the colored parts represent the newly added components that handle the generation of resources.

In order to be able to apply the CVAE to business processes, we had to adapt it by employing encoder and decoder architectures suitable for handling both sequential data (activities, timestamps and resources) and non-sequential data (trace attributes). In particular, for sequential data, we drew inspiration from seq2seq models, which are commonly used in NLP, where an encoder model maps a variable-length input sequence to a fixed size vector, which is then “unrolled” back to a variable-length sequence by a decoder model (Sutskever et al. 2014).

The following sections explain the conditional variable, the preprocessing steps, the encoder and decoder architectures, the training approach, and the generation process of new traces.

Conditional variable

The conditional variable c is a value or vector used by the model both in the encoder and the decoder, during both training and generation. Consequently, it must be present in the log (or must be deterministically obtained from it) in order to be used by the model during training. At generation time, it must be explicitly provided to the decoder to generate traces conditioned on the specified value.

In our domain, the conditional variable represents a characteristic of the trace. Thus, it is a trace-level variable that remains constant across all events of a trace.

The conditional variable can be a single value or a vector. It may represent either a numerical or categorical variable (e.g., encoded as a one-hot vector), or even a combination of both. For instance, in predictive process mining, the conditional variable could correspond to the trace label in an outcome-based prediction task. In “Data-sets” section we define the conditional variables for the dataset considered in the evaluation.

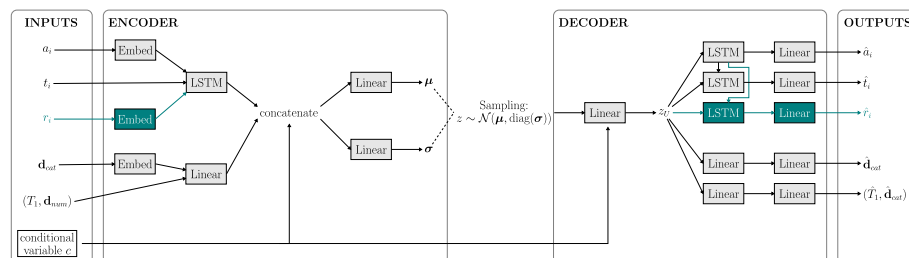


Fig. 2 A high level overview of the CVAE for process mining. Activities a_i and event interarrival times $t_i = T_i - T_{i-1}$ are incrementally processed by the LSTM subnet, while all the categorical $\mathbf{d}_{cat}$ and numerical $\mathbf{d}_{num}$ attributes of x (including trace arrival time T_1) are processed by the Linear subnet

As discussed in “[Conditional variational autoencoders \(CVAEs\)](#)” section, the model learns different distributions for each value of the conditional variable, enabling the generation of new traces based on a specified condition.

Preprocessing

In the preprocessing phase a trace x is properly manipulated to be feed to the encoder.

The possible values for each categorical variable (i.e. activities, resources and categorical attributes) are sorted alphabetically and mapped to an index. For example, *Activity_A* may be mapped to 0, *Activity_B* may be mapped to 1, and so on. This mapping is performed to prepare categorical values to be embedded by the encoder, as explained in “[Encoder](#)” section.

Event timestamps T_i are decomposed into two parts: (i) the trace arrival time T_1 ; which corresponds to the timestamp of the first event in the trace, and is handled by the model in the same way as a numerical trace attribute; and (ii) the *event interarrival time* t_i which is defined as $t_i := T_i - T_{i-1}$ for $i > 1$ with $t_1 = 0$ as a special case. The *event interarrival time* is normalized using the 95 th percentile value instead of the maximum value. While this causes some values to be outside the $[0, 1]$ range, we found it useful in practice to ignore outlier timestamps, i.e. very high interarrival time, that could cause the normalization to squash *event interarrival times* to an exceedingly narrow range close to zero.

Numerical trace attributes are preprocessed by normalizing them to the $[0, 1]$ interval using min-max normalization.

Encoder

The goal of the encoder is to map any trace x and conditioning variable c to the mean vector $\mu_{x,c}$ and variance vector $\sigma_{x,c}$ that fully specify the multivariate Gaussian distribution $q_\phi(\mathbf{z}|x, c) \equiv \mathcal{N}(\mu_{x,c}, \text{diag}(\sigma_{x,c}))$, with a diagonal covariance matrix, for the latent variables $\mathbf{z}$ (the subscripts in $\mu_{x,c}$ and $\sigma_{x,c}$ will be omitted whenever the dependency of these parameters on x and c is clear from the context).

First, in the encoder each categorical variable is passed through its own embedding layer and transformed into a numerical vector. The embedding layer is an end-to-end learned lookup table that maps indices to vectors of arbitrary dimension, which is typically used in language models (Bengio et al. 2003). Using embedding vectors instead of indices or one-hot encoding offers several advantages in handling categorical variables; an explanation of the rationale behind this choice and an experimental study of their impact in this work can be found in Appendix A.

Then, the encoder consists of two paths: the first makes use of an LSTM layer to handle activities, timestamps and resources, the second employs a fully connected layer for each trace attribute. Thereafter, outputs of the two paths are concatenated together, the conditional variable is added, and lastly two fully connected layers are used to map to mean μ and variance σ vectors.

Decoder

In the training process, after sampling a latent space vector $\mathbf{z}$ from the Gaussian distribution $\mathcal{N}(\mu_{x,c}, \text{diag}(\sigma_{x,c}))$ identified by the encoder, the goal of the decoder is to generate a trace from $\mathbf{z}$ that is as similar as possible to the original trace x . At generation time,

instead, the decoder is provided with a vector $\mathbf{z}$ sampled from the prior latent distribution $\mathcal{N}(\vec{0}, \mathbf{I})$ and a value of the conditioning variable c , in order to generate a realistic trace x aligned with c .

As mentioned above, the decoder is in charge of approximating the true conditional distribution $p(x|\mathbf{z}, c)$ of the execution traces x of the form $x = (\mathbf{d}_{num}, \mathbf{d}_{cat}) \otimes \langle e_1, \dots, e_n \rangle$ of length n , with $e_i = (a_i, T_i, r_i)$ for $i \in \{1, \dots, n\}$. Assuming that the event sequence is conditionally independent (given $\mathbf{z}$ and c) of the static trace attributes³ and using the above-described time decomposition $T_i = T_{i-1} + t_i$, this distribution can be unfolded as follows:

$$p(x | \mathbf{z}, c) = p(\mathbf{d}_{num}, \mathbf{d}_{cat}, T_1 | \mathbf{z}, c) \cdot \prod_{i=1}^n p(e_i | e_{<i}, \mathbf{z}, c)$$

where $e_{<i} = \langle e_1, \dots, e_{i-1} \rangle$, while factor $p(\mathbf{d}_{num}, \mathbf{d}_{cat}, T_1 | \mathbf{z}, c)$ is the probability of observing, given $\mathbf{z}$ and c , the trace arrival time T_1 and the values $\mathbf{d}_{num}$ and $\mathbf{d}_{cat}$ for the trace attributes, if any⁴.

In its turn, the “per-event” joint probability $p(e_i|\mathbf{z}, c)$ can be unfolded as: $p(e_i|\mathbf{z}, c) = p(a_i, t_i, r_i|\mathbf{z}, c) = p(a_i|\mathbf{z}, c) \cdot p(r_i|a_i, \mathbf{z}, c) \cdot p(t_i|a_i, r_i, \mathbf{z}, c)$. However, since we empirically observed that conditioning t_i on r_i does not bring advantages, we decided to replace $p(t_i|a_i, r_i, \mathbf{z}, c)$ with the simpler term $p(t_i|a_i, \mathbf{z}, c)$ —further alternative factorizations, which we explored before choosing this one, are discussed later on.

In the proposed framework, a decoder neural network with parameters θ is trained to learn an approximated version $p_\theta(x|\mathbf{z}, c)$ of the above-described probabilistic model.

First of all, since we are in a conditional setting, the conditional variable c is concatenated to the latent vector $\mathbf{z}$. Then, a fully connected layer upsamples $\mathbf{z}$ to a higher dimension vector $\mathbf{z}_U$. This approach, even though it has been criticized in previous NLP works (Bowman et al. 2016), has proven effective in our case to improve decoder performances. Activities, timestamps and resources are obtained from $\mathbf{z}_U$ using three different autoregressive LSTM schemes, where different kinds of information are explicitly provided as input at each time step, as a complement to the current hidden state value:

1. At each time step i , the *activity LSTM* takes as input both $\mathbf{z}_U$ and the previously reconstructed activity $\hat{a}_{i-1}$ (for the first time step, a special Start Of Trace (SOT) token is used) and outputs the current activity $\hat{a}_i$.
2. The *timestamp LSTM* takes as input, in addition to $\mathbf{z}_U$ and the previous event interarrival time $\hat{t}_{i-1}$, also the current reconstructed activity $\hat{a}_i$, and outputs the current event interarrival time $\hat{t}_i$.
3. Similarly, the *resource LSTM* takes as input, in addition to $\mathbf{z}_U$ and the previously reconstructed resource $\hat{r}_{i-1}$, also the current reconstructed activity $\hat{a}_i$, and outputs

³ This assumption, common in the literature, is mild, since we are hypothesizing that events sequence and case attributes are independent of one another in an absolute/global fashion, but only conditionally to the latent variable $\mathbf{z}$ (i.e., once fixing/known the value of $\mathbf{z}$). Any dependence between a case attribute a and an event e_i of a trace can be captured indirectly via the latent variable $\mathbf{z}$, the actual value of which can influence the realization of the random variables representing a and e_i .

⁴ For the sake of simplicity, in the current implementation of our framework, trace attributes (including T_1) are regarded as conditionally independent of one another, so that this factor reduces to a product of single-attribute conditional probabilities.

the current event resource $\hat{r}_i$. Here, $\hat{a}_i$ serves to constrain $\hat{r}_i$ to the pool typically responsible for executing that activity, while $\hat{r}_{i-1}$ supports the reproduction of work handover patterns—i.e., how resources transfer the execution of activities between each other.

Different configurations has been tested, namely (i) not using the latent space as input for each time step, (ii) not conditioning the timestamp LSTM to the current activity $\hat{a}_i$, (iii) conditioning the timestamp LSTM also on the current resource $\hat{r}_i$, (iv) using a shared LSTM for both activities and timestamps, (v) not conditioning the resource LSTM to the current activity $\hat{a}_i$. However, the first configuration yielded poor reconstructed control flows, the second, third and fourth configurations resulted in poor timestamp reconstruction and the fifth configurations produced inaccurate resources.

Categorical and numerical attributes reconstruction follows different paths, one for each attribute, composed of a sequence of two fully connected layers and ReLU activations. For both activity and categorical attribute predictions, the model outputs a probability distribution for each possible value. During inference, the argmax operator is used to select the most likely value, while during training, this is replaced by the softmax function to ensure differentiability.

Training

We train the model end-to-end with backpropagation, optimizing the CVAE loss function (4). In particular, the reconstruction loss of a reconstructed trace $\hat{x} \sim p_\theta(x|\mathbf{z}, c)$ with respect to its ground-truth trace x is the sum of the following loss components:⁵

- Binary Cross Entropy (BCE) loss of each trace activity a_i
- Mean Squared Error (MSE) loss of each event interarrival time t_i
- BCE loss of each event resource r_i
- BCE loss of each categorical trace attribute in $\mathbf{d}_{cat}$
- MSE loss of each numerical trace attribute in $\mathbf{d}_{num}$

To prevent vanishing of the KL divergence loss we make use of a technique called *KL cyclical annealing*, which varies β following a linear cyclical schedule as training progresses (Fu et al. 2019). When $\beta < 1$, the model is able to focus more on improving reconstruction. In our tests without cyclical annealing, the KL divergence loss decreased to nearly zero whereas the reconstruction loss did not improve much.

Generation

After the model has been trained, new traces can be generated by randomly sampling a latent space vector $\mathbf{z}$ from the multivariate standard Gaussian distribution $\mathcal{N}(\vec{0}, \mathbf{I})$, concatenating a conditional variable to it and feeding the resulting vector to the decoder

⁵ Using these reconstruction loss components corresponds to evaluating the data log-likelihood by comparing each categorical trace attribute against the categorical (softmax-normalized) distribution predicted for it by the decoder and every numerical attribute against a Gaussian distribution located at the respective real-valued prediction returned by the decoder.

network. The conditional variable makes it possible to limit the generation of only those traces associated with the chosen value of the conditional variable.

The decoder activity LSTM recurrently generates activities for the trace until a special End Of Trace (EOT)⁶ token gets generated or until a fixed maximum trace length is reached. For each activity, the LSTM outputs a probability distribution and the argmax operator is used to choose the most likely activity.

Similarly, the resource LSTM outputs a probability distribution and the argmax operator is used to choose the most likely resource.

To reconstruct timestamps, the decoder makes use of three pieces of information, namely (1) an arbitrary start timestamp τ , (2) the *trace arrival time* $\hat{T}_1$ and (3) the *event interarrival times* $\hat{t}_i$. The timestamp $\hat{T}_i$ of activity $\hat{a}_i$ is computed as follows: $\hat{T}_i \leftarrow \tau + \hat{T}_1 + \sum_{k=1}^i \hat{t}_k$. The start timestamp τ can be chosen arbitrary, but it is usually set to the first timestamp of the entire log, or to the first or last timestamp of the test log. Also note that, given the way trace arrival times T_i are processed, the model generates traces that follow the arrival time distribution of the training set.

Evaluation

In this section, we present the evaluation methodology used to assess the performance of our conditional generative models for trace generation. We compare our method with other state-of-the-art generative models. Additionally, to ensure that the extensions to our method do not compromise the quality of previously generated features, we also compare it with its earlier version as described in Graziosi et al. (2024). We aim to address the following research questions:

- RQ1 How does our method perform in terms of control flow and temporal dimension compared to other state-of-the-art generative models?
- RQ2 How does our method perform in terms of the quality of the generated resources compared to other state-of-the-art generative methods which generate roles?
- RQ3 How does our method perform in terms of the quality of the generated attributes—resources and trace attributes—in terms of distribution compared to the original process?

RQ1 investigates the overall quality of the generated traces through a comprehensive analysis that considers multiple perspectives which were already included in the analysis of (Graziosi et al. 2024). Specifically, it compares our method to both the earlier version of our model and other state-of-the-art generative models. More specifically, to answer RQ1, the following research questions are investigated:

- RQ1.1 How does our method perform in generating traces close to the ones of the original process in terms of control flow and temporal fidelity?
- RQ1.2 How does our method perform in terms of the trade-off between the variability of the generated traces and conformance with the original process?

⁶ In our implementation, the Start of Trace (SOT) and End of Trace (EOT) tokens coincide.

RQ1.3 How does our method perform in terms of the effectiveness of the control provided by the conditional variable over the generation process?

The analysis hence focuses on: (i) the fidelity of generated traces across the following perspectives: control flow, temporal event distribution, and trace duration (RQ1.1); (ii) the variability of the generated traces, evaluated through coverage (the model's ability to reproduce distributions from the original log) and generalization (its ability to produce novel, unseen traces while adhering to process rules) (RQ1.2); and (iii) the effectiveness of conditional generation, i.e., the model's ability to correctly generate traces aligned with the specified conditional variables (RQ1.3).

The second and third research questions focus on evaluating the quality of the newly extended capability of our model: the generation of categorical event attributes, specifically resource features. RQ2 evaluates the quality of the generated resource features by comparing our method with the only other generative model capable of generating resource-related data, the approach based on Camargo et al. (2019). However, since this method generates only resource roles rather than individual resources, the comparison is conducted at the role level. RQ3 examines the distribution of trace attributes and individual resources across the generated traces—features not offered by other generative methods. Since no direct comparison is possible, the evaluation is conducted against the distributions in the original event log, which serve as a benchmark to assess the quality of our model's output.

Datasets

Our evaluation considers four examples based on three publicly available real-world event logs, preprocessed to remove incomplete traces. For each example, we define a binary conditioning on trace execution, that distinguishes between two relevant subprocesses, enabling us to generate only the desired traces by tuning the conditional variable. Moreover, for simplicity in presenting the results, we select a single trace attribute for each dataset, choosing the one we consider most significant for each process.

- **Sepsis cases** (Mannhardt et al. 2016) contains events of sepsis cases from a hospital. We filtered out incomplete traces with a missing “Release” activity. We consider the patient's *age* as the main trace attribute. We consider the following conditional labelling: the patient has a relapse and returns to the emergency room within 28 days from discharge. We denote this example as **Sepsis**.
- **BPIC2012** (van Dongen 2012) contains a loan application process. We filtered out incomplete traces and those corresponding to ineligible applications, keeping those for which at least one offer was created⁷. We consider the loan *requested amount* as the main trace attribute. We consider two different conditional labellings for this log: (i) the reject of a loan offer by the applicant, indicated by the presence of the activity “O_DECLINED-COMPLETE”, denoted as **Bpic2012_a**; (ii) the creation of multiple

⁷ This is a common choice in the literature (Teinmaa et al. 2019). The excluded traces primarily consist of short traces with one or two activities, which represent invalid applications that are automatically desk rejected. By filtering these, we focus on more interesting and complex traces for generation.

Table 2 Datasets description

Dataset	Traces	Variants	Activities	Resources # (Roles)	Avg. trace length	Avg. trace cycle time	Total duration	Conditional ratio
Sepsis	782	733	16	25 (7)	17	38.2 d	~ 1.5yrs	14%
Bpic2012_a	4685	3790	36	63 (11)	40	18.7 d	166 d	17%
Bpic2012_b	4685	3790	36	63 (11)	40	18.7 d	166 d	28%
Traffic_Fines	129615	200	10	148 (3)	4	20.2wks	~ 13yrs	4%

loan offers by the bank, indicated by the presence of multiple “O_CREATED-COM-
PLETE” events, denoted as **Bpic2012_b**⁸.

- **Traffic fines** (de Leoni and Mannhardt 2015) is an event log of an information system managing road traffic fines. We filtered out incomplete traces, whose last activity is “Send Fine”. We consider the *amount* of the fine as the main trace attribute. The conditional labelling is defined by the presence of an appeal request by the offender to the judge or the prefecture. This corresponds to the presence of any of the following activities: “Appeal to Judge”, “Send Appeal to Prefecture”, “Insert Data Appeal to Prefecture”, “Notify Result Appeal to Offender”, “Receive Result Appeal from Prefecture”. This example is denoted as **Traffic_Fines**.

In Table 2, we report details on the four datasets, highlighting their variety in terms of number of traces, activity labels, cycle times, total log duration, and *conditioning ratios*, i.e., the percentage of traces for which the conditional labelling is true.

Experimental setup

Each dataset is obtained by adding a trace attribute to every trace, containing the value of the conditional labelling, as defined in the previous section. Each dataset is then split in training, validation and test set in chronological order with ratios 60%–20%–20%^{9,10}.

Our model is trained using the hyperparameters reported in Table 3.

During training, the validation set is used to activate an early stopping mechanism, which stops the training when the loss function, computed on the validation set, does not improve for 500 epochs (not counting epochs for which the loss is worse than the current best loss, but still within a certain range, configured through the *Early stop delta* parameter), and outputs the model with the best loss. This only applies when evaluating the full loss function ($\beta = 1$, i.e., KL annealing cycles are excluded from early stopping). While an early stop patience of 500 might seem high, we chose it for two different

⁸ The BPIC2012 log also contains information about the events’ lifecycle transitions: “SCHEDULE”, “START”, and “COMPLETE”. This information is concatenated with the activity label of each event in order to reproduce it.

⁹ For the **Traffic_Fines** dataset, we use only a 5% test set, which contains 6480 traces, comparable in order of magnitude to the other test logs.

¹⁰ The 60%-20%-20% split differs from the 70%-10%-20% split used in Graziosi et al. (2024), chosen to enlarge the validation set and achieve a more robust early stopping mechanism.

Table 3 Hyperparameters

Hyperparameter	Value
Embedding size (activities)	5
Embedding size (resources)	5
Embedding size (categorical attributes)	5
LSTM hidden size	200
Latent space size	10
Learning rate	3×10^{-4}
Dropout	5%
Batch size	256
Max number of epochs	20000
Early stop patience	500
Early stop delta	5% ^a
Number of KL annealing cycles	8

^a For flexibility, i.e. to be able to use the same value for multiple logs with different loss magnitudes, we chose the early stop delta parameter to be 5% of the current best validation loss

reasons: first, from previous trainings, we noticed that the **cvae** was able to eventually minimize the validation loss even after hundreds of epochs of worsening performance¹¹, so we did not want to halt the training process too soon with a lower value of patience. Moreover, since we are applying the KL annealing cycles technique and the loss function changes during training, we believe more time should be granted to the model to improve the loss.

The trained model is used to generate 10 different logs, which are used for the assessment. Each generated log contains the same number of traces as the corresponding test set. The generation is guided by setting the conditional variable to reproduce the same conditional ratio found in the training log.

Metrics

To evaluate the quality of the generative models we adopt a subset of the metrics introduced in Chapela-Campa et al. (2025). These metrics are designed to assess the ability to replicate the observed behavior of the process across different perspectives, including control-flow, time, and resource. All metrics are computed by comparing the generated log with the test log.

For the control-flow evaluation, we employ the following metrics: *Control-Flow Log Distance* (CFLD), which minimizes the total distance by pairing the traces from the generated and test logs using the Hungarian algorithm (Kuhn 1955), and *N-Gram Distance* (NGD). Specifically, we adopt the 2GD, which evaluates pairs of consecutive activities (2-grams) in the generated log compared to those in the test set. For all control-flow metrics, lower values indicate greater similarity between the generated and real logs.

CLFD and NGD metrics assess the capability of our approach to reproduce the activity sequences within each trace and their overall distribution across the entire log. In particular, CLFD is more precise and sensitive to long-term dependencies due to the

¹¹ In Graziosi et al. (2024), the early stopping mechanism was set to 100 epochs. We noticed that the addition of the reconstruction loss for the resources made the training task more challenging, requiring more epochs for convergence.

complete comparison between traces, whereas NGD focuses on local relationships by analysing the distribution of N-grams across the log.

To evaluate the temporal perspective, we select two metrics: *Cycle Time Distribution* (CTD), which measures the distance between the cycle time distributions of all traces, and *Relative Event Distribution* (RED), which analyzes the temporal distribution of events relative to the case origin (i.e., the case arrival).

CTD is a general metric that assesses the ability to reproduce the entire cycle time of traces in the process. In contrast, RED focuses on evaluating the reconstruction of events (and their timestamps) from the beginning to the end of the corresponding case. Both metrics are independent of when the traces enter the system.

For the resource perspective, we employ the *Circadian Workforce Distribution* (CWD). By dividing the process timeline into days of the week, this metric determines the average difference in the allocation of active resources. Specifically, it detects potential differences in the number of active resources and/or the availability calendars associated to them, throughout each day of the week¹².

For all time and resource metrics, the comparison between the generated log and the test set is performed by comparing two histograms derived from them, which are then transformed into distribution functions. These distributions are compared using the *1 st Wasserstein Distance* (1 WD), a computationally efficient variation of the Earth Mover's Distance (EMD). Even for time and resource metrics, the lower the value, the closer the generative log is to the real one.

In order to assess the *conformance* and *variability* of generated traces, we consider *Conformance Score* (CONF), inspired by the work in Buliga et al. (2023), and analyze the variants of generated logs. CONF measures the average number of DECLARE constraints (Pesic et al. 2007) satisfied by the generated traces. The DECLARE constraints are mined from the entire event log with a support of 90%¹³. CONF ranges from 0 to 1, where the maximum value indicates that the generated log is perfectly conformant to the real log. The variant analysis examines how many of the generated variants are present in the training or test logs. A high proportion of generated variants appearing in the training log may indicate that the model is capturing the underlying data distribution, but an excessively high value could signal overfitting to the training data. Conversely, the presence of unseen generated variants in the test log, or generated variants with high CONF, demonstrates that the model achieves good generalization.

Finally, to assess the control over the conditional generation, we compute the *actual conditional ratio* of the generated traces. This is done by recomputing the conditional labelling a posteriori, as defined in Sect. 6.1, and comparing the conditional ratio of the generated log with those of the training and test sets. Since all methods are trained to reproduce the conditional ratio of the training set, generative models that consistently

¹² Since the **Traffic Fines** log does not include time-of-day information in its timestamps, we slightly adjusted the CWD metric computation for this dataset. Instead of computing separate distributions for each weekday, we use a single distribution representing the average daily resource allocation over the entire week. This preserves the original intent of the metric while ensuring meaningful comparisons.

¹³ To ensure fairness, only generated variants that do not already appear in the training log are considered for this analysis.

generate logs with a similar ratio to the training log demonstrate effective control over the generation.

Baselines

In this section, we introduce the baselines used for the comparative analysis of our model. For the control flow and temporal perspective, we consider the following two generative methods as baselines:

- The LSTM-based model introduced in Camargo et al. (2019), denoted as **lstm**;¹⁴
- *ProcessGAN*, a GAN-based model presented in Li et al. (2024), denoted as **p_gan**.

Our choice is justified by the following reasons: **lstm** has been identified as a state-of-the-art deep learning generative model in Camargo et al. (2021) and continues to be used as baseline as the reference DL generative method in many recent works (Camargo et al. 2022; Meneghello et al. 2023). **p_gan** is a recent example of GAN-based generative method that leverages several advanced techniques, as detailed in “Related work” section¹⁵.

Since both of these methods are not aware of the conditional variable¹⁶, we employ each of them in the following two ways:

- For each method, we train a single model on the full training set and use it to unconditionally generate traces. We denote these methods as **lstm1** and **p_gan1**, respectively;
- For each method, we train two separate models on the two subsets of the training data identified by the value of the binary conditional variable. We then generate traces using both models to reproduce the conditional ratio of the training set. We denote these methods as **lstm2** and **p_gan2**, respectively.

We denote our method as **cvae**. To assess the effect of the integration of resource generation into the model, we also consider the earlier version of our model based on Graziosi et al. (2024), which does not generate resources. We denote this version as **cvae_old**¹⁷.

The **p_gan** models automatically generate a number of traces equal to that in the training set¹⁸. So to compare with the test log, we split the generated log into three parts.

¹⁴ We use a more recent implementation of the method from Camargo et al. (2019), as presented in Ali et al. (2024), which introduces new strategies for sampling the next activity in suffix prediction tasks. For generation, we follow the approach in Camargo et al. (2021) and apply the random strategy for next activity sampling. The hyperparameter optimization is performed over 10 epochs.

¹⁵ Two relevant GAN-based methods for next event and suffix prediction, respectively, are Taymouri et al. (2020) and Taymouri et al. (2021). However their adaptation to the generative task is not straightforward. Therefore we select (Li et al. 2024), which is explicitly designed for the generation task, as the reference GAN-based method.

¹⁶ *ProcessGAN* (Li et al. 2024) is defined as a conditional model, but the conditional variable therein is primarily used to enhance the temporal distribution of trace durations and not for the conditional generation.

¹⁷ The hyperparameters used to train **cvae_old** are those described in Graziosi et al. (2024), differing from those in Table 3 only in the value of the early stopping patience parameter, which is set to 100. Additionally, the model is trained on the split 60%-20%-20%, consistent with all other methods, which differs from the split used in Graziosi et al. (2024).

¹⁸ A configuration parameter, `seq_num`, is available to specify the number of generated traces. However, in our experiments, changing `seq_num` to a value different from the number of traces in the training set resulted in an error during the generation process.

Table 4 Details of models and baselines

Model	Reference	Generated logs #	Activities	Timestamps	Resources/ Roles	Conditional generation
cvae	–	10	✓	✓	✓ (Resources)	✓
cvae_old	Graziosi et al. (2024)	10	✓	✓	✗	✓
lstm1	Ali et al. (2024)	10	✓	✓	✓ (Roles)	✗
lstm2	Ali et al. (2024)	10	✓	✓	✓ (Roles)	✓
p_gan1	Li et al. (2024)	3	✓	✓	✗	✗
p_gan2	Li et al. (2024)	3	✓	✓	✗	✓
train_log	–	3	✓	✓	✓ (Resources)	✗

For all other models (**cvae**, **cvae_old**, and the **lstm** models), we generate ten logs, each with the same size of the test log. This implies that the results for the **p_gan** models will have weaker statistics than those of the other models. Moreover, we were unable to train **p_gan** on the **Traffic_Fines** log. Likely due to the log size, which is in the order of tens of thousands of traces, the **p_gan** training terminated unexpectedly with an out-of-memory error. For this reason, results for **p_gan** on **Traffic_Fines** will not be available in “**Results**” section.

Lastly, we also consider the original log itself as a baseline, denoted as **train_log**. This serves as an additional baseline for the metrics CLFD, 2GD, RED, CTD, CONF, and CWD. To achieve this, we split the training set into 3 splits (12 splits for **Traffic_Fines**) based on the chronological order. Each of these splits contains the same number of traces as the test set, thus enabling a direct comparison when computing the aforementioned metrics.

In Table 4, we summarize all the relevant information about our model and the baselines introduced in this section.

Results

In this section, we analyze and discuss the results obtained from the evaluation of our method, **cvae**¹⁹. The discussion is structured according to the research questions formulated in “**Evaluation**” section.

Answering RQ1

In this section, we provide an answer to the first research question (RQ1), which investigates the quality of the generated traces from the control flow and temporal perspectives. This evaluation builds on the analysis presented in Graziosi et al. (2024).

Control flow and temporal fidelity (RQ1.1)

First, we analyse the fidelity of the generated traces in terms of control flow, timestamps and trace duration (RQ1.1). This evaluation assesses how closely the generated traces align with those of the real process appearing in the test log.

¹⁹ The implementation to reproduce the experiments and the datasets, can be accessed at the following link: <https://github.com/rgraziosi-fbk/cvae-process-mining>.

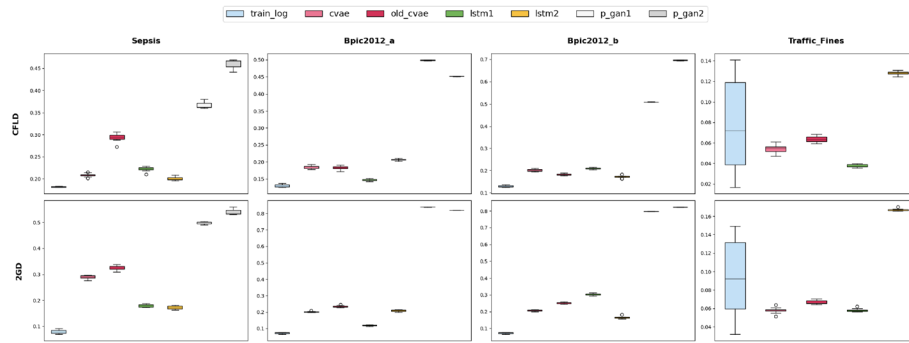


Fig. 3 Control flow metrics: CFLD and 2GD. Lower metric values indicate greater similarity between the generated log and the test log

Control flow In Fig. 3, we present the results of the two key metrics for analysing the *control flow* fidelity of the generated logs compared to the test log: CFLD and 2GD (see “Metrics” section). CLFD provides a more comprehensive measure of control flow distance, while 2GD focuses more narrowly on the distribution of transitions between pairs of activities in the log.

We begin by comparing our method, **cvae**, with its previous version, **cvae_old**. With the exception of the CFLD metric in the **Bpic2012_b** dataset, our new version consistently improves control-flow fidelity compared to its predecessor. However, the differences between the two methods are generally small, apart from a notable improvement in the CFLD metric for the **Sepsis** log. We hypothesize that the improvement may stem from the updated early stopping mechanism, which in **cvae_old** was fixed at 100 epochs, while in the current version, it is set to 500 epochs²⁰.

When comparing our method with other state-of-the-art methods, we first observe that both versions of **p_gan** perform worse than all other methods across both metrics and all datasets where they were applied. This suggests that **p_gan** generates traces with relatively low quality in the control-flow perspective.

Conversely, the two **lstm** versions perform significantly better among the other baselines, often competing with or outperforming our method. For the CFLD metric, performance differences are generally small. **lstm1** outperforms our method in two cases, with the largest margin being 0.04, and is outperformed in the other two cases, with a largest margin of 0.01. Similarly, **lstm2** surpasses our method in two examples, with a maximum margin of 0.03, while our method outperforms it in the other two examples, with the largest margin being 0.07. Regarding the 2GD metric, the differences are slightly more pronounced. **lstm1** outperforms our method in two examples, with a maximum margin of 0.11, is outperformed in one case with a margin of 0.1, and ties in the remaining example. Similarly, **lstm2** outperforms our method in two cases, with the largest margin being 0.12, is outperformed in one case with a margin of 0.11, and ties in the remaining example.

²⁰ The decision to increase the value for the early stopping mechanism is driven by the higher number of epochs required for the convergence of the more complex loss function.

Overall, these results indicate that our method reproduces the control-flow perspective of the generated traces with quality comparable to the state-of-the-art non-conditional generative methods available in the literature.

Temporal distribution and trace duration In Fig. 4, we present the two metrics, RED and CTD, which evaluate the temporal event distribution within traces and the trace duration distribution, respectively (see “Metrics” section).

First of all, we notice that **cvae** almost always outperforms its previous version, **cvae_old**, in both metrics. Only in the **Sepsis** experiment their mean performances are comparable, although we can observe greater variability in the older method. As in the case of the control flow analysis, we attribute this to the different settings of the early stopping mechanism in the two methods.

In the comparison with other generative models, **cvae** consistently outperforms its competitors in both the RED and CTD metrics across all experiments, with a significant margin. Additionally, the performance of **cvae** is closer than the other methods to that of the baseline **train_log**.

These results indicate that our model demonstrates excellent fidelity in reproducing the temporal perspective of the generated traces. It accurately captures the distribution of events within traces as well as trace durations, producing traces that closely resemble those in the test log. The distances between the generated logs and the test log are comparable to those between the training and test logs, indicating that the generated traces fall within the natural variability of the original process.

Variability and conformance (RQ1.2)

We assess now the trade-off between variability of the generated traces and their conformance to the process rules.

Variability To provide an indication of the variability of the traces generated from a control flow perspective, Table 5 reports the mean number of trace variants produced by each method across datasets, specifying how many of these variants are already present in the training and test logs, respectively.

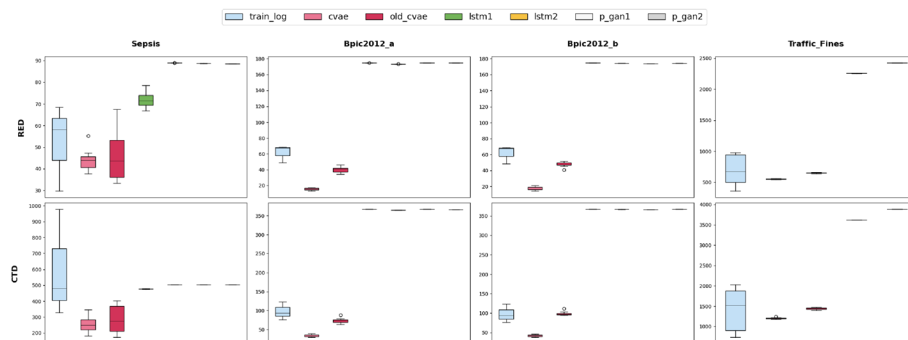


Fig. 4 Temporal metrics: RED and CTD. Lower metric values indicate greater similarity between the generated log and the test log

Table 5 For each dataset we report the number of variants in the train ($\mathcal{V}_{\text{train}}$) and test ($\mathcal{V}_{\text{test}}$) log, and for every model the average number of generated variants ($\mathcal{V}_{\text{gen}}$) detailing how many are also present in the each of the original logs

Dataset	$\mathcal{V}_{\text{train}}$	$\mathcal{V}_{\text{test}}$	Model	$\mathcal{V}_{\text{gen}}$	$\mathcal{V}_{\text{gen}} \cap \mathcal{V}_{\text{train}}$	$\mathcal{V}_{\text{gen}} \cap \mathcal{V}_{\text{test}}$
Sepsis	452	152	cvae	128	6	0
			cvae_old	116	5	0
			lstm1	157	0	0
			lstm2	157	3	1
			p_gan1	157	0	0
			p_gan2	151	0	0
Bpic2012_a	2374	864	cvae	912	39	18
			cvae_old	875	68	33
			lstm1	911	64	35
			lstm2	860	63	37
			p_gan1	937	0	0
			p_gan2	937	0	0
Bpic2012_b	2374	864	cvae	915	53	24
			cvae_old	844	50	26
			lstm1	927	10	8
			lstm2	921	26	15
			p_gan1	929	0	0
			p_gan2	826	0	0
Traffic_Fines	160	46	cvae	69	46	25
			cvae_old	30	18	16
			lstm1	142	36	23
			lstm2	233	26	20
			p_gan1	N/A	N/A	N/A
			p_gan2	N/A	N/A	N/A

Since we generate logs with the same size as the test log, we expect the number of generated variants to be similar to that of the test log. This expectation holds true for most generative models. In **Sepsis**, both **cvae** and **cvae_old** generate slightly fewer variants compared to the other methods, which are closer to the number of variants found in the test log. In **Bpic2012_a** and **Bpic2012_b**, all models generate a similar number of variants to those in the test log. In contrast, for the **Traffic_Fines** dataset, both **cvae** and **cvae_old** generate a number of variants closer to that of the test log, while **lstm1** and **lstm2** generate a much larger number of variants, three and five times the number of variants in the test log, respectively.

These results show that while all the methods demonstrate similar capabilities in generating variability, **cvae** and **cvae_old** are more reliable in reproducing variability that closely mirrors that of the original process.

Table 6 Conformance metric (CONF). Higher values indicate greater adherence to process constraints. Bold values indicate the generative method achieving the highest conformance for each dataset

dataset	train_log	cvae	cvae_old	lstm1	lstm2	p_gan1	p_gan2
Sepsis	98.7%	98.4%	97.5%	91.2%	94.9%	60.2%	56.9%
Bpic2012_a	99.9%	97.3%	95.6%	98.4%	98.7%	48.8%	68.3%
Bpic2012_b	99.9%	97.2%	82.1%	93.7%	94.7%	48.5%	39.8%
Traffic_Fines	100%	100%	100%	78.2%	93.6%	N/A	N/A

Coverage Coverage reflects the ability of the generative model to replicate the distribution of the original data. Table 5 reports the number of generated variants that overlap with those in the training and test logs, providing an indication of this property²¹.

Before analysing the data in Table 5, we offer the following observation. Table 2 shows that the **Sepsis**, **Bpic2012_a**, and **Bpic2012_b** logs exhibit high variability in control flow, as evidenced by the number of trace variants being close to the total number of traces. This suggests that the generated logs would likely replicate a relatively low number of variants from the original logs. In contrast, the **Traffic_Fines** log has significantly fewer variants with respect to the number of traces. For this dataset, we would expect the generative models to replicate a higher proportion of variants observed in the original log.

Indeed, these expectations are confirmed by the results in Table 5. For **cvae** and **cvae_old**, in the first three datasets, only about 5% of the generated traces are present in the training log, while, for the **Traffic_Fines** dataset, this percentage increases to approximately 60% for both methods. **lstm1** and **lstm2** show comparable coverage to **cvae** in the first three datasets. However, in the **Traffic_Fines** dataset, their coverage drops significantly compared to **cvae**, achieving only 25% and 11%, respectively. This is directly linked to the higher absolute number of variants generated compared to the test log, as discussed in the previous paragraph. Finally, the **p_gan** methods perform the worst, with zero coverage across all datasets.

This analysis highlights the ability of **cvae** and **cvae_old** to maintain a balance between variability and coverage, especially in datasets with differing levels of control flow complexity, underscoring their robustness compared to other generative models.

Conformance and generalization Conformance assesses the degree to which the generated traces align with the process rules. Table 6 presents the results of the CONF metric, which quantifies the average proportion of generated traces that comply with the process rules mined from the event log (see “Metrics” section). To ensure fairness, only generated variants that do not already appear in the training log are used for this analysis.

²¹ While it may seem that generating already existing variants is less valuable for a generative model, the inclusion of additional data attributes, such as temporal information, often results in traces that are unique and not identical to those in the original log.

We first observe that the conformance of **cvae** is consistently very high across all datasets, with values always above 97%. Only in the **Bpic2012_a** dataset, **lstm1** and **lstm2** slightly outperform **cvae**. However, these two methods do not maintain the same level of conformance across all datasets. Specifically, they achieve conformance rates of 91% and 95%, respectively, in **Sepsis**, and only 78% and 94% in **Traffic_Fines**.

We also observe that the previous version of our method, **cvae_old**, exhibits less stable conformance results across datasets, with its lowest conformance rate being 82% in the **Bpic2012_b** dataset. This variation, compared to **cvae**, can once again be attributed to the different early stopping mechanisms implemented in the two models.

The two **p_gan** models instead achieve the lowest conformance scores across all datasets, which aligns with their poor performance in the control flow analysis. This suggests that these models not only fail to capture the control flow distribution of the original process but also often generate traces that violate the process rules.

Finally, we can discuss the *generalization* capability of our model, which refers to its ability to generate original, unseen yet meaningful samples. Based on the results from Table 5 and the conformance performance discussed earlier, we can conclude that **cvae** demonstrates excellent generalization. It achieves good variability while maintaining very high conformance. In contrast, although the **lstm** models generate higher variability than **cvae**, they do not reach the same level of conformance, indicating a lower generalization capability.

Conditional generation (RQ1.3)

In this section, we evaluate the generative models' ability to perform conditional generation, focusing on their capability to produce traces that conform to predefined conditions or constraints.

As described in “[Metrics](#)” section, to evaluate the control over conditional generation, we recompute the conditional labeling a posteriori, following its definition in “[Datasets](#)” section, and assess the conditional ratios of the generated logs against those of the training and test logs, as per the evaluation methodology defined in “[Experimental setup](#)” section. We configure the generation to match the conditional ratio observed in the training log, except for **lstm1** and **p_gan1** methods that do not support controlled generation. Table 7 summarizes the conditional ratios for the generated logs alongside the original ratios in the training and test logs. This comparison

Table 7 Conditional ratios for the training, test, and generated logs. Bold values indicate the models with conditional ratios closest to those of the training log for each dataset. The difference from the training log ratio is shown in parentheses

dataset	train %	test %	cvae %	cvae_old %	lstm1 %	lstm2 %	p_gan1 %	p_gan2 %
Sepsis	14.5	11.5	15.2 (+0.7)	17.0 (+2.5)	42.2 (+27.7)	36.5 (+22)	47.3 (+32.8)	9.6 (−4.9)
Bpic2012_a	16.0	19.2	12.5 (−3.5)	13.0 (−3)	29. (+13.6)	9.8 (−6.2)	14.3 (−1.7)	40.4 (+24.4)
Bpic2012_b	26.8	26	19.8 (−7)	17.8 (−9)	29.4 (+2.6)	28.0 (+1.2)	15.8 (−11)	15.9 (−10.9)
Traffic_Fines	3.0	3.6	3.2 (+0.2)	3.2 (+0.2)	4.4 (+1.4)	3.1 (+0.1)	N/A	N/A

evaluates how accurately the generative models reproduce the constraints applied during inference and highlights any discrepancies in label reproduction. Ideally, the generated logs should closely match the conditional ratios of the training set.

Although **cvae** achieves the closest match to the training log conditional ratio only in the **Sepsis** dataset, it performs consistently well across the other datasets, with deviations ranging from 0.2% to 7% from the training log ratio. Its previous version, **cvae_old**, performs slightly worse, with deviations between 0.2% and 9%, but still maintains stable results.

In contrast, none of the other methods demonstrate similar stability. For example, **lstm2** achieves the closest ratios to the training log in the **Bpic2012_b** and **Traffic_Fines** datasets, but performs poorly in the other datasets, with deviations ranging from up to 22%.

Finally, **p_gan1** produces the closest ratio in the **Bpic2012_a** dataset; however, given its poor performance in other aspects discussed earlier, this result is likely coincidental. Additionally, **p_gan1** exhibits the largest deviation from the training ratio across all methods and datasets, with the greatest deviation of 32.8% observed in the **Sepsis** dataset.

This analysis demonstrates that our model effectively controls the generation process, producing traces that adhere to specific conditions with acceptable accuracy. Notably, it surpasses the performance of **lstm2**, which consists of two separate models (see “[Base-lines](#)” section), each learning to reproduce traces corresponding to a specific condition label.

Answering RQ2

In this section, we address the second research question (RQ2), which examines the quality of the generated resource information compared to other state-of-the-art generative methods.

Specifically, the only other generative model in the literature that generates resource information is **lstm**, which provides only the role of the resource. Therefore, our comparison focuses on the level of roles (see “[Resources and roles](#)” section). To ensure consistency, we use the same grouping of resources into roles as applied in the **lstm** model, which clusters resources with similar activity profiles based on the method described in Song and van der Aalst (2008).

Figure 5 presents the CWD metric values (see “[Metrics](#)” section) calculated for role allocation distributions, denoted as CWD_ROLE. This metric captures the average number of roles active at different hours throughout each weekday. Across all datasets, **cvae** outperforms both **lstm** models, achieving CWD values very close to those of

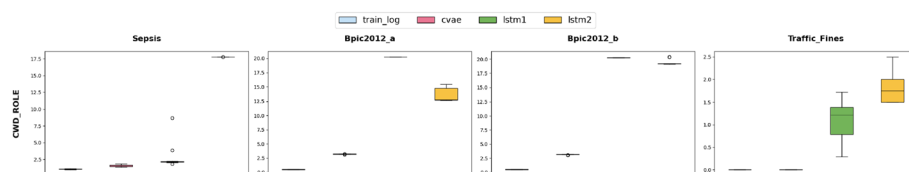
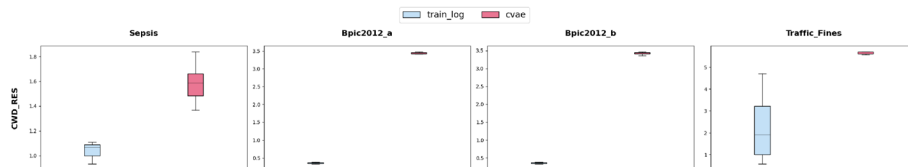


Fig. 5 Resource metrics: CWD_ROLE computed for resource role distributions. Lower values indicate greater similarity between the role distributions in the generated log and the test log

Table 8 Allocation rate of roles in the **Traffic_Fines** dataset

Role	<i>train_log</i>	<i>cvae</i>	<i>lstm1</i>	<i>lstm2</i>
role 1	99.7%	99.7%	82.3%	86.3%
role 2	0.273%	0.264%	4.01%	0.650%
role 3	0.0270%	0.0360%	13.7%	13.5%

**Fig. 6** Resource metrics: CWD_RES computed for individual resource distributions. Lower values indicate greater similarity between the resource distributions in the generated log and the test log

train_log on all datasets. The **lstm** models perform significantly worse, with the exception of **lstm1** in the **Sepsis** dataset, where its performance is comparable to that of **cvae**. In the **Traffic_Fines** dataset, we observe that CWD_ROLE is zero for **cvae**, as it is for the **train_log**. This is because, among the three roles present in this dataset (see Table 2), a single role is used in more than 99% of all events in the original log. As a result, the role allocation distributions become trivial, leading to a zero CWD metric. This behavior is accurately captured by **cvae**, whereas the **lstm** models fail to replicate it, as shown in Table 8, which reports the overall allocation rate of each role in the original log and in the generated ones.

These results highlight the ability of **cvae** to accurately reproduce resource role distributions, achieving superior performance compared to other generative models producing the same type of information. Moreover, the role distributions generated by **cvae** closely align with those in the original process across the majority of logs. This consistency across diverse datasets underscores the robustness of **cvae** in modeling resource allocation effectively, making it a reliable choice for applications requiring accurate and stable role generation.

Answering RQ3

In this section, we evaluate the quality of the generation of individual resources and trace attributes. Since no other generative models generate this information, the evaluation will consist of a quantitative and qualitative comparison with the original process log. We separate the discussion into resources and trace attributes.

Resources We assess the quality of the resource distributions generated by the **cvae** model through the CWD metric (see “Metrics” section), calculated for individual resources in the generated log, and compare it with the baseline **train_log**.

Figure 6 shows the CWD metric values for individual resource allocation distributions, denoted as CWD_RES. A low CWD value, close to that of the **train_log**, indicates good generation performance, as the generated resources are allocated in alignment with the

original log allocation distribution. The results indicate that the difference between the **cvae** model and the **train_log** is very small (around 0.5) for the **Sepsis** dataset, and more pronounced for the two **Bpic2012** datasets (around 3) and for the **Traffic_Fines** dataset (around 4). These outcomes can be understood in relation to the number of resources in each dataset, as detailed in Table 2. Specifically, the **Sepsis** dataset, which has the fewest resources, exhibits the smallest CWD values, while **Bpic2012** and **Traffic_Fines** datasets, which include a larger number of resources, have higher CWD values, reflecting their greater complexity and challenge for accurate resource generation.

To further assess the quality of resource generation, we analyze the allocation frequencies of each resource in the original logs compared to those in the logs generated by **cvae**. Figure 7 presents this comparison, providing insights into how well the model replicates the original resource distribution patterns. A closer resemblance to the original data suggests better performance in capturing resource allocation dynamics.

For each dataset, we conduct two comparisons: one against the training log (on the left) and one against the test log (on the right). As shown in Fig. 7a, c, e, g, the generated resource frequency distributions closely match those of the training logs. This indicates that the model successfully learns and reproduces resource usage patterns from the training data.

When comparing with the test logs, the results vary. In the **Sepsis** dataset (Fig. 7b), the generated distribution aligns well with the test log. However, in the two **Bpic2012** datasets (Fig. 7d, f), alignment is slightly weaker. While the model correctly captures the most frequently used resources, discrepancies emerge in the allocation of less frequent ones. Additionally, some resources appear in the test log but not in the generated one, and vice versa.

The observed differences arise from the evolving nature of resource availability—new resources may be introduced while others phase out over time. This effect is especially noticeable in the **Traffic_Fines** dataset (Fig. 7h), where a significant mismatch between the distributions in the generated and test logs is evident. Many resources present in the test log do not appear in the training log, and consequently, are absent in the generated logs as well. This issue is more pronounced in the **Traffic_Fines** dataset due to its extensive temporal span of approximately 13 years (see Table 2), during which the set of available resources naturally evolved²².

In conclusion, the resources generated by the **cvae** model effectively replicate the resource frequencies patterns observed in the training log, as shown in Fig. 7. Furthermore, the generated log reasonably reproduces the overall resource allocation distribution over time in comparison to the test log, as indicated by the relatively small difference in the CWD_RES metric, presented in Fig. 6, between **cvae** and **train_log**.

Trace attributes Finally, we assess the quality of trace attribute generation across the different logs. As described in “*Datasets*” section, we select the most relevant trace attribute for each dataset. Figure 8 illustrates the frequency distributions of these trace attributes in the generated logs compared to those in the test log.

²² This is the main reason why the CWD metric in Fig. 6 is computed based on cumulative resource usage distribution over time, rather than individual resource distributions.

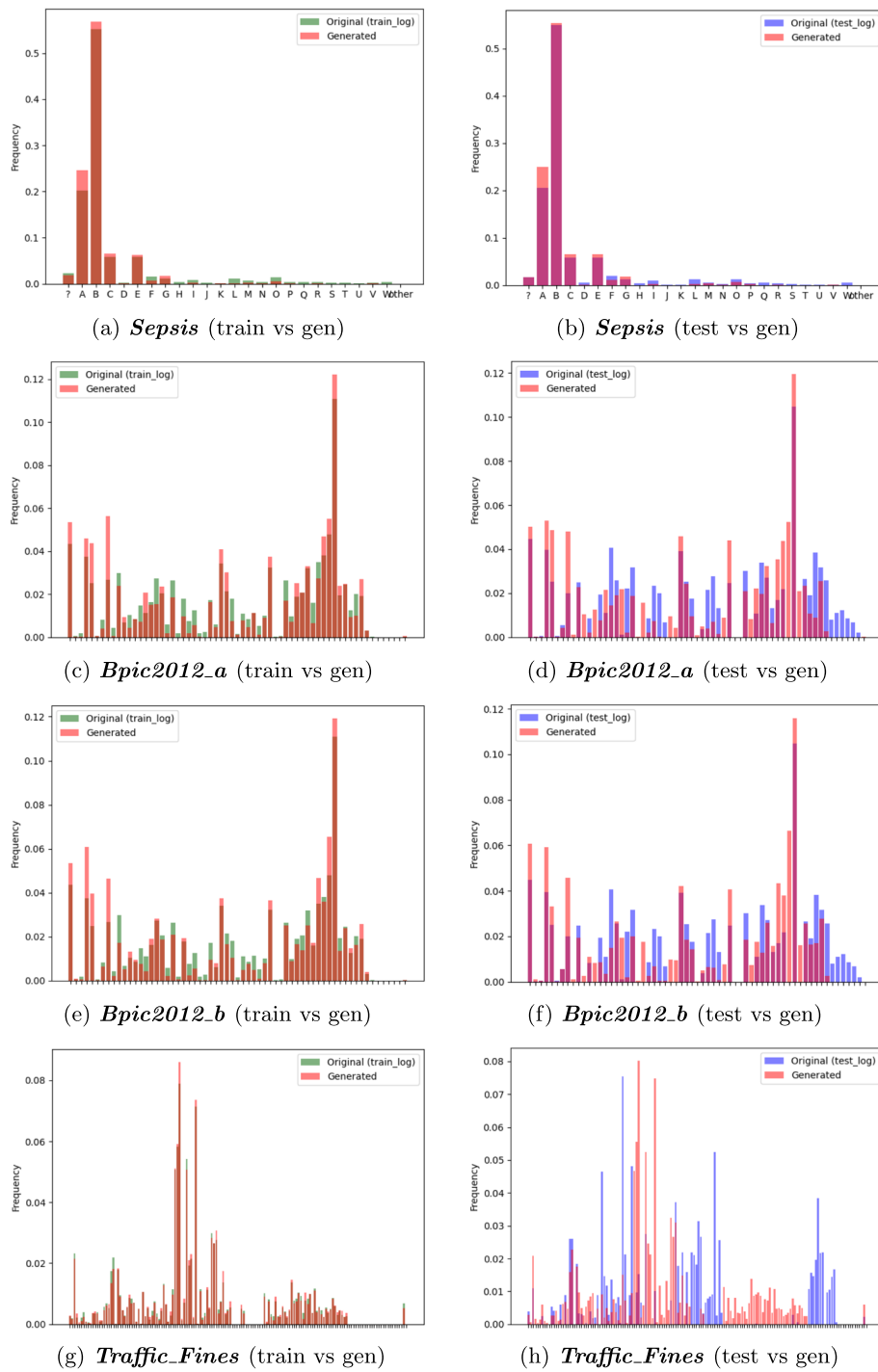


Fig. 7 Resource distributions of original train and test logs compared with the **cvae** generated log

The effectiveness of trace attribute generation can be assessed by comparing how closely the distributions of generated attributes resemble those of the real test log in Fig. 8. In particular, a closer resemblance suggests better reproduction of the original process characteristics.

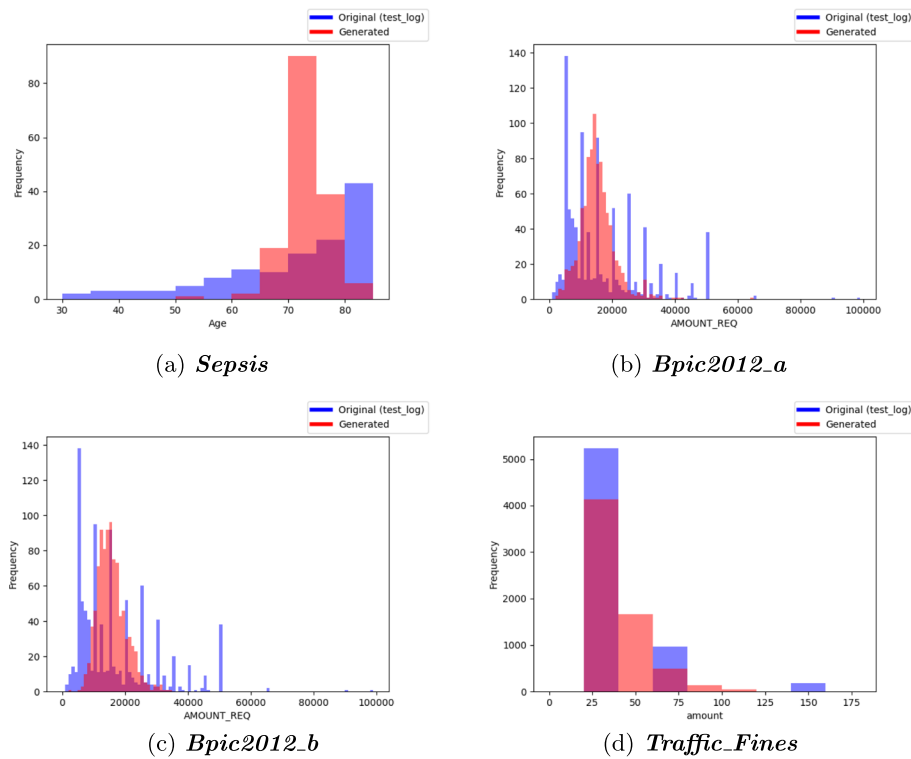


Fig. 8 Comparison of the trace attribute value distributions between the original test log and the log generated by **cvae**

The results indicate that the trace attributes are generated with distributions differing from those of the original process. Specifically, they tend to exhibit a more “Gaussian-like” shape, concentrating most of the values around a central mean. While the generated trace attribute values remain meaningful, they fail to adequately capture the diversity of values present in the original log. An exception is observed in the case of the **Traffic_Fines** dataset (Fig. 8d), where the *amount* attribute shows a good reproduction of the original distribution.

This analysis highlights the challenge of accurately reproducing trace attribute distributions, and suggests that further refinement of the generative model may be required to better capture the nuances of the original data.

Discussion and limitations

Through various analyses, we have demonstrated the effectiveness of our model in generating multi-perspective process traces that closely replicate many aspects of the original process. Specifically, our model exhibits strong fidelity in reproducing the control-flow, temporal dynamics, and resource allocation, while achieving reasonably good results for trace attribute values. In these aspects, our approach competes with and often outperforms other state-of-the-art deep learning generative methods.

A key strength of our model lies in its ability to strike a balance between the variability of the generated traces and their conformance to process rules. Compared to alternative approaches, our method excels in maintaining this trade-off, ensuring that the generated

traces are both diverse and aligned with the constraints of the process. Moreover, our model leverages its conditional architecture to achieve effective and stable control over the generation process. This approach proves superior to the expedient of training separate models for each type of trace to be generated, which is not only less stable but also impractical for scenarios involving multi-valued or continuous conditional variables. By contrast, our model offers a more flexible and scalable solution for generating conditional traces across a variety of scenarios.

Our analysis has also highlighted certain criticalities that concern the quality of the generated traces. In particular, the analysis performed for answering RQ3 highlights the challenge of generating data payloads, particularly trace attributes, and resources. Concerning trace attributes, the evaluation shows that the generated attributes are meaningful and process-relevant. Nonetheless, Fig. 8 shows that their distributions, in the generated traces, tend to be more regular and smoothed compared to those in the original logs. In contrast, the original process exhibits more varied and irregular patterns, suggesting that improvements may be needed to fully capture the diversity and nuances of trace attribute distributions.

Concerning resources, the results in Figs. 6 and 7 show that our model, by generating the sequence of resource usage within each trace, is able to reproduce the overall distribution observed in the real process. This is a non-trivial result that can be attributed to the model's ability to learn the resource handover patterns of the original process and leverage this information to replicate the distribution of resource usage. Notably, such patterns have already been explicitly exploited in the literature for event log simulation (Kirchdorfer et al. 2024). However, despite this overall alignment, Fig. 7 also reveals local mismatches between the generated resources and those in the test set. As already stated in "Answering RQ3" section this is mainly due to the evolving nature of resource availability and resource turnover between the training and test logs. This effect is more pronounced in datasets with long time spans, where resource availability naturally evolves over time. The significance of this issue depends on the objective of the generation. If the goal is to augment historical data, such differences are not problematic. However, if the aim is to generate data that closely reflects the actual resource profile at a specific point in time, they may become more relevant. This consideration can be generalized to other trace characteristics, and may not concern just resources. This problem may be addressed using a continuous conditional variable that represents the temporal position of a trace within the log's time horizon. By selecting traces based on this variable, we could generate data more representative of a specific time period. While this solution is already feasible in our model, this in-depth evaluation goes beyond the aims of this paper and we have not yet conducted structured experiments in this direction. Nonetheless, we intend to pursue this investigation in future works.

Limitations

The main limitations affecting the validity of the evaluation carried out are external validity threats, limiting the generalization of the results. First, our evaluation focuses on just four different datasets. Nonetheless, they exhibit different characteristics and can be considered representative of typical event logs in the Process Mining domain. Importantly, all datasets were derived from real-world event logs, which naturally include a

certain degree of noise. This ensures that the evaluation reflects realistic conditions and demonstrates that our model can operate effectively even in the presence of such noise. We attribute this robustness to regularization techniques such as dropout and early stopping, which help prevent overfitting. A more systematic investigation of the model's robustness to noise would, however, be a valuable direction for future work.

Second, we have focused our evaluation on the simpler case of binary conditional variables. However, we believe that our choice of conditional variables covers a broad set of scenarios commonly encountered in Process Mining. An extensive evaluation involving more complex conditional variable definitions—such as multiclass and continuous variables—would be highly valuable but falls beyond the scope of this work and would require a dedicated investigation.

A final limitation of our evaluation is the lack of a structured analysis on the impact of hyperparameter choices, particularly those related to early stopping. The parameters controlling the early stopping mechanism can significantly affect both training duration and the balance between *coverage* and *generalization* of the generated data. A more loose early stopping criterion allows the model to train longer, potentially leading to overfitting. While this can improve certain metrics assessing similarity between generated traces and the original dataset it may come at the cost of reduced diversity in the generated traces. That is, while the generated log may still generalize to some extent (as our evaluation is conducted on the test set), an excessively long training period could result in a model that overly replicates patterns seen in the training set rather than capturing broader variations. We mitigated this issue by selecting early stopping parameters that provided a good balance between generalization and coverage, as shown in Section RQ1.2. However, a systematic study of this aspect would be an interesting direction for future research.

Addressing these limitations will be crucial for enhancing the fidelity and versatility of our method, enabling it to generate multi-perspective process traces that more accurately reflect the complexities of real-world processes.

Conclusions and future directions

In this paper, we introduced a conditional variational autoencoder for multi-perspective process trace generation. We assessed our model on various datasets against multiple baselines and demonstrated its effectiveness in producing meaningful and relevant multi-perspective process traces. Our results showed that the proposed method outperforms competitors in terms of the quality of traces' control-flow, cycle time, event temporal distribution, resources, and trace attributes, while also ensuring compliance with the implicit constraints of the process. Additionally, we demonstrated that the conditional architecture allows robust control over the generation process, enabling the creation of traces tailored to specific conditions or the simulation of "what-if" scenarios.

Looking forward, we aim to address some of the current weaknesses of our model, particularly in: (i) improving the generation of trace attributes to enhance its fidelity and versatility; (ii) exploring the use of the conditional variable to better align generated resources with resource turnover; and (iii) investigating how different hyperparameter settings, such as early stopping, influence the trade-off between training time, coverage, and generalization, in order to refine our model's performance.

Additionally, we could explore enhancing the control flow generation by adopting more sophisticated activity sampling techniques than the argmax function, such as beam search, which could enable the production of more accurate and diverse trace sequences. Another ambitious direction for future research is to extend the generation from individual traces to sets of concurrent traces. Such an approach would enable the accurate modeling of inter-case dynamics, such as resource contention, process capacity, trace inter-arrival times, and other phenomena that arise from interactions between multiple process instances.

Appendix A ablation study on embedding layers

In this section, we present an ablation study to validate the effectiveness of using the embedding layer in our model.

In our model (**cvae**), categorical variables are embedded with an embedding layer (see “**Encoder**” section), which maps indices to n -dimensional vectors. Embeddings have shown promising properties in NLP (Bengio et al. 2003), because they avoid the curse of dimensionality problem caused by one-hot encoding, and also they tend to map semantically similar categorical values to nearby positions in the embedding n -dimensional vector space. Inspired by other works in predictive process monitoring and generative models in process mining (Evermann et al. 2016; Camargo et al. 2022), we believe the same benefits may also apply to categorical process data, which is why our model makes use of embeddings for categorical variables.

To validate this hypothesis, we designed an ablation study in which we trained a variant of our model with the embedding layers disabled, where categorical variables were encoded using one-hot encoding instead. In the following, we denote this variant as **cvae_onehot**. Table 9 presents the results of the comparative evaluation between the base **cvae** model and the **cvae_onehot** model.

In general, **cvae** outperforms or ties **cvae_onehot** across all metrics. In particular, in the few instances where **cvae_onehot** is better, the difference is quite small; on the other hand, **cvae** surpasses **cvae_onehot** by a large margin in some metrics (e.g., CONF of **Sepsis**, RED and CTD for **Bpic2012_a** and **Bpic2012_b**).

Table 9 Evaluation of **cvae** (which employs embedding layers) against **cvae_onehot** (which makes use of one-hot encoding). Each model was used to generate 10 logs, so values in the table are averages

Dataset	Model	CONF	CFLD	2GD	RED	CTD	CWD_ROLE	CWD_RES
Sepsis	cvae	98.4%	0.208	0.290	43.9	252	1.56	1.58
	cvae_onehot	94.9%	0.358	0.377	42.7	308	1.58	1.59
Bpic2012_a	cvae	97.4%	0.185	0.202	15.3	34.4	3.20	3.44
	cvae_onehot	97.5%	0.304	0.271	76.5	114	3.17	3.44
Bpic2012_b	cvae	97.2%	0.201	0.205	17.9	41.6	3.16	3.43
	cvae_onehot	96.0%	0.234	0.202	43.9	77.8	3.21	3.44
Traffic_Fines	cvae	100%	0.0540	0.0580	554	1200	0	5.67
	cvae_onehot	100%	0.0560	0.0570	559	1223	0	6.17

The metrics that improve the most by using embeddings are control-flow (CONF, CFLD, 2GD) and temporal metrics (RED, CTD), whereas resource metrics (CWD_ROLE, CWD_RES) do not show significant improvements.

While it may initially seem strange that temporal metrics benefit from embeddings (given that timestamps are not embedded), we attribute this to the fact that timestamp reconstruction depends on the ability of the encoder to compress the trace as a whole into a latent space vector. This ability, in turn, depends not only on timestamps but also on activities and resources, which do make use of embeddings. In short, embedding layers in the encoder indirectly influence timestamp reconstruction in the decoder through the latent space.

On the other hand, for most logs (namely **Sepsis**, **Bpic2012_a** and **Bpic2012_b**) metrics related to roles and resources, i.e. CWD_ROLE and CWD_RES, do not benefit from embeddings. This may seem surprising at first, especially since other dimensions, such as control flow, improve significantly with embedding layers. However, since the CWD metric only considers the number of active resources and not the specific resources being allocated (see “Metrics” section), an improvement in resource allocation due to embeddings would not be reflected in an improvement of the metric. The other advantage of embeddings is dimensionality reduction, and its positive impact can be observed in **Traffic_Fines**, which has the highest number of resources among all logs (see Table 2): in **Traffic_Fines**, embedding resources results in a significant 8% reduction in CWD_RES.

When using embeddings, the **Traffic_Fines** log shows little to no improvement in control-flow and temporal metrics: this may be explained by its high number of traces and control-flow simplicity. On the other hand, **Sepsis** contains a relatively low number of traces and has a complex control-flow: using embeddings resulted in significantly better temporal and control-flow metrics. Conversely, role/resource metrics improved considerably in **Traffic_Fines** (which has the highest number of resources) thanks to embeddings, and not at all in the other logs (which have fewer resources). Together, these observations suggest that embeddings yield greater benefits when certain process data dimensions (either control flow, temporal or resource dimension) is scarce or complex, whereas their impact is limited or even negligible when data is abundant or simple.

Unexpectedly, **Bpic2012_a** and **Bpic2012_b**, which differ only by the conditional variable definition, show different gains in control-flow metrics: **Bpic2012_a** improves similarly to **Sepsis**, whereas **Bpic2012_b** control-flow metrics do not improve much.

Finally, **Bpic2012_a** and **Bpic2012_b** show the highest gains in temporal metrics when employing embeddings, probably because of their long average trace length.

Authors' contributions

M.R. project leader, scientific direction, R. G. implementation leader, A. B., F. F., F.M. implementation of baselines, C. G., F. D. F., L. P. scientific contribution and direction. All authors contributed to write and review the manuscript.

Funding

We acknowledge the support of the PNRR project FAIR - Future AI Research (PE00000013), under the NRRP MUR program funded by the NextGenerationEU, the HORIZON 2020, and the PRIN project PINPOINT Prot. 2020 FNEB27, CUP H23 C22000280006 and H45E2100021000.

Data availability

Data and the code repository link (<https://github.com/rgraziosi-fbk/cvae-process-mining>) is provided within the manuscript.

Declarations

Ethics approval and consent to participate

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 16 December 2024 Accepted: 8 May 2025

Published online: 20 May 2025

References

- Adamo G, Borgo S, Di Francescomarino C, Ghidini C, Guarino N, Sanfilippo EM (2017) Business processes and their participants: An ontological perspective. In: AI*IA 2017 Advances in Artificial Intelligence - XVIth International Conference of the Italian Association for Artificial Intelligence, Bari, Italy, November 14–17, 2017, Proceedings, Springer, Berlin-Heidelberg, LNCS, vol 10640, pp 215–228. https://doi.org/10.1007/978-3-319-70169-1_16
- Agarwal P, Gupta A, Sindhgatta R, Dechu S (2022) Goal-oriented next best activity recommendation using reinforcement learning. CoRR abs/2205.03219. <https://doi.org/10.48550/ARXIV.2205.03219>
- Ali MA, Dumas M, Milani F (2024) Enhancing the accuracy of predictors of activity sequences of business processes. In: Research Challenges in Information Science - 18th International Conference, RCIS 2024, Guimarães, Portugal, May 14–17, 2024, Proceedings, Part I, Springer, Berlin-Heidelberg, Lecture Notes in Business Information Processing, vol 513, pp 149–165. https://doi.org/10.1007/978-3-031-59465-6_10
- Bengio Y, Ducharme R, Vincent P, Janvin C (2003) A neural probabilistic language model. *J Mach Learn Res* 3:1137–1155. <https://jmlr.org/papers/v3/bengio03a.html>
- Bose RPJC, van der Aalst WMP, Žliobaitė I, Pechenizkiy M (2014) Dealing with concept drifts in process mining. *IEEE Trans Neural Netw Learn Syst* 25(1):154–171. <https://doi.org/10.1109/TNNLS.2013.2278313>
- Bowman SR, Vilnis L, Vinyals O, Dai AM, Józefowicz R, Bengio S (2016) Generating sentences from a continuous space. In: Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning, CoNLL 2016, Berlin, Germany, August 11–12, 2016, ACL, Kerrville, pp 10–21. <https://doi.org/10.18653/V1/K16-1002>
- Buliga A, Di Francescomarino C, Ghidini C, Maggi FM (2023) Counterfactuals and ways to build them: Evaluating approaches in predictive process monitoring. In: Advanced Information Systems Engineering - 35th International Conference, CAISE 2023, Zaragoza, Spain, June 12–16, 2023, Proceedings, Springer, Berlin-Heidelberg, Lecture Notes in Computer Science, vol 13901, pp 558–574. https://doi.org/10.1007/978-3-031-34560-9_33
- Camargo M, Dumas M, González-Rojas O (2020) Automated discovery of business process simulation models from event logs. *Decis Support Syst* 134:113284. <https://doi.org/10.1016/j.dss.2020.113284>
- Camargo M, Dumas M, Rojas OG (2021) Discovering generative models from event logs: data-driven simulation vs deep learning. *PeerJ Comput Sci* 7:e577. <https://doi.org/10.7717/peerj-cs.577>
- Camargo M, Dumas M, Rojas OG (2022) Learning accurate business process simulation models from event logs via automated process discovery and deep learning. In: Advanced Information Systems Engineering - 34th Int. Conf., CAISE 2022, Proc., Springer, Berlin-Heidelberg, LNCS, vol 13295, pp 55–71. https://doi.org/10.1007/978-3-031-07472-1_4
- Camargo M, Dumas M, Rojas OG (2019) Learning accurate LSTM models of business processes. In: Business Process Management - 17th Int. Conf., BPM 2019, Proc., Springer, Berlin-Heidelberg, LNCS, vol 11675, pp 286–302. https://doi.org/10.1007/978-3-030-26619-6_19
- Chapela-Campa D, Bencheikroun I, Baron O, Dumas M, Krass D, Senderovich A (2023) Can I trust my simulation model? measuring the quality of business process simulation models. In: Business Process Management - 21st Int. Conf., BPM 2023, Proc., Springer, Berlin-Heidelberg, LNCS, vol 14159, pp 20–37. https://doi.org/10.1007/978-3-031-41620-0_2
- Chapela-Campa D, Bencheikroun I, Baron O, Dumas M, Krass D, Senderovich A (2025) A framework for measuring the quality of business process simulation models. *Inf Syst* 127(102):447. <https://doi.org/10.1016/j.is.2024.102447>
- de Leoni M, Mannhardt F (2015). Road traffic fine management process. <https://doi.org/10.4121/UUID:270FD440-1057-4FB9-89A9-B699B47990F5>
- Di Francescomarino C, Ghidini C (2022) Predictive process monitoring. In: Process Mining Handbook, LNBIP, Springer International Publishing, Berlin-Heidelberg, pp 320–346. https://doi.org/10.1007/978-3-031-08848-3_10
- Evermann J, Rehse J, Fettke P (2016) A deep learning approach for predicting process behaviour at runtime. In: Business Process Management Workshops - BPM 2016 International Workshops, Rio de Janeiro, Brazil, September 19, 2016, Revised Papers, Springer, Berlin-Heidelberg, LNBIP, vol 281, pp 327–338. <https://doi.org/10.1007/978-3-319-58457-7>
- Evermann J, Rehse JR, Fettke P (2017) Predicting process behaviour using deep learning. *Decis Support Syst* 100:129–140. <https://doi.org/10.1016/j.dss.2017.04.003>
- Fu H, Li C, Liu X, Gao J, Celikyilmaz A, Carin L (2019) Cyclical annealing schedule: A simple approach to mitigating KL vanishing. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2–7, 2019, Volume 1 (Long and Short Papers), ACL, Kerrville, pp 240–250. <https://doi.org/10.18653/V1/N19-1021>
- Graziosi R, Ronzani M, Buliga A, Di Francescomarino C, Folino F, Ghidini C, Meneghello F, Pontieri L (2024) Generating the traces you need: A conditional generative model for process mining data. In: 2024 6th International Conference on Process Mining (ICPM), IEEE, Piscataway, pp 25–32. <https://doi.org/10.1109/ICPM63005.2024.10680621>
- Hinton GE, Salakhutdinov RR (2006) Reducing the dimensionality of data with neural networks. *Science* 313(5786):504–507. <https://doi.org/10.1126/science.1127647>

- Jalayer A, Kahani M, Beheshti A, Pourmasoumi A, Motahari-Nezhad HR (2020) Attention mechanism in predictive business process monitoring. In: 24th International Enterprise Distributed Object Computing Conference (EDOC), IEEE, Piscataway, pp 181–186. <https://doi.org/10.1109/EDOC49727.2020.00030>
- Ketykó I, Mannhardt F, Hassani M, van Dongen BF (2022) What averages do not tell: predicting real life processes with sequential deep learning. In: SAC '22: The 37th ACM/SIGAPP Symposium on Applied Computing, Virtual Event, April 25–29, 2022, ACM, New York, pp 1128–1131. <https://doi.org/10.1145/3477314.3507179>
- Kingma DP, Welling M (2014) Auto-encoding variational bayes. In: 2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14–16, 2014, Conference Track Proceedings. [arXiv:1312.6114](https://arxiv.org/abs/1312.6114)
- Kirchdorfer L, Blümel R, Kampik T, Van der Aa H, Stuckenschmidt H (2024) Agentsimulator: An agent-based approach for data-driven business process simulation. In: 2024 6th International Conference on Process Mining (ICPM), pp 97–104. <https://doi.org/10.1109/ICPM63005.2024.10680660>
- Klijn I, Mannhardt F, Fahland D (2024) Multi-perspective concept drift detection: Including the actor perspective. In: Advanced Information Systems Engineering - 36th International Conference, CAISE 2024, Limassol, Cyprus, June 3–7, 2024, Proceedings, Springer, Berlin-Heidelberg, LNCS, vol 14663, pp 141–157. https://doi.org/10.1007/978-3-031-61057-8_9
- Krajsic P, Franczyk B (2021) Variational autoencoder for anomaly detection in event data in online process mining. In: Proc. of the 23rd Int. Conf. on Enterprise Information Systems, ICEIS 2021, Volume 1, SCITEPRESS, Setúbal, pp 567–574. <https://doi.org/10.5220/0010375905670574>
- Kuhn HW (1955) The hungarian method for the assignment problem. *Nav Res Logist Q* 2(1–2):83–97. <https://doi.org/10.1002/nav.3800020109>
- Li K, Yang S, Sullivan TM, Burd RS, Marsic I (2024) ProcessGAN: Generating privacy-preserving time-aware process data with conditional generative adversarial nets. *ACM Trans Knowl Discov Data* 18(9). <https://doi.org/10.1145/3687464>
- Lin L, Wen L, Wang J (2019) Mm-pred: A deep predictive model for multi-attribute event sequence. In: Proceedings of the 2019 SIAM International Conference on Data Mining, SDM 2019, Calgary, Alberta, Canada, May 2–4, 2019, SIAM, Philadelphia, pp 118–126. <https://doi.org/10.1137/1.9781611975673.14>
- Mannhardt F (2018) Multi-perspective process mining. In: Proceedings of the Dissertation Award, Demonstration, and Industrial Track at BPM 2018 co-located with 16th International Conference on Business Process Management (BPM 2018), Sydney, Australia, September 9–14, 2018. CEUR Workshop Proceedings, vol 2196. CEUR-WS.org, Aachen, pp 41–45
- Mehdiyev N, Evermann J, Fettke P (2017) A multi-stage deep learning approach for business process event prediction. In: 19th IEEE Conference on Business Informatics, CBI 2017, Thessaloniki, Greece, July 24–27, 2017, Volume 1: Conference Papers, IEEE Computer Society, Piscataway, pp 119–128. <https://doi.org/10.1109/CBI.2017.46>
- Mannhardt F (2016). Sepsis cases - event log. <https://doi.org/10.4121/UUID:915D2BFB-7E84-49AD-A286-DC35F063A460>
- Meneghello F, Di Francescomarino C, Ghidini C (2023) Runtime integration of machine learning and simulation for business processes. In: 5th Int. Conf on Process Mining, ICPM 2023, IEEE, Piscataway, pp 9–16. <https://doi.org/10.1109/ICPM60904.2023.10271993>
- Nolle T, Seeliger A, Mühlhäuser M (2018) Binet: Multivariate business process anomaly detection using deep learning. In: Business Process Management - 16th Int. Conf., BPM 2018, Proc., Springer, Berlin-Heidelberg, LNCS, vol 11080, pp 271–287. https://doi.org/10.1007/978-3-319-98648-7_16
- Object Management Group BPMN Technical Committee et al (2011) Business process model and notation, version 2.0. OMG, Object Management Group INeedham, MA
- Pesic M, Schonenberg H, van der Aalst WM (2007) Declare: full support for loosely-structured processes. In: 11th IEEE International Enterprise Distributed Object Computing Conference (EDOC 2007), p 287. <https://doi.org/10.1109/EDOC.2007.14>
- Sharma P, Kumar M, Sharma HK, Biju SM (2024) Generative adversarial networks (GANs): Introduction, Taxonomy, Variants, Limitations, and Applications. *Multimed Tools Appl*. <https://doi.org/10.1007/s11042-024-18767-y>
- Song M, van der Aalst WM (2008) Towards comprehensive support for organizational mining. *Decis Support Syst* 46(1):300–317. <https://doi.org/10.1016/j.dss.2008.07.002>
- Sutskever I, Vinyals O, Le QV (2014) Sequence to sequence learning with neural networks. In: Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, MIT Press, Cambridge, pp 3104–3112. <https://proceedings.neurips.cc/paper/2014/hash/a14ac55a4f27472c5d894ec1c3c743d2-Abstract.html>
- Sani MF, Gonzalez JJG, van Zelst SJ, van der Aalst WMP (2020) Conformance checking approximation using simulation. In: 2nd Int. Conf. on Process Mining, ICPM 2020, IEEE, Piscataway, pp 105–112. <https://doi.org/10.1109/ICPM49681.2020.00025>
- Sohn K, Lee H, Yan X (2015) Learning structured output representation using deep conditional generative models. In: Advances in Neural Information Processing Systems 28:3483–3491. <https://proceedings.neurips.cc/paper/2015/hash/8d55a249e6baa5c06772297520da2051-Abstract.html>
- Tax N, Teinemaa I, van Zelst SJ (2018) An interdisciplinary comparison of sequence modeling methods for next-element prediction. *Softw Syst Model* 19:1345–1365. <https://doi.org/10.1007/S10270-020-00789-3>
- Tax N, Verenich I, Rosa ML, Dumas M (2017) Predictive business process monitoring with LSTM neural networks. In: Advanced Information Systems Engineering - 29th International Conference, CAISE 2017, Essen, Germany, June 12–16, 2017, Proceedings, vol 10253, pp 477–492. https://doi.org/10.1007/978-3-319-59536-8_30
- Taymouri F, Rosa ML, Erfani SM (2021) A deep adversarial model for suffix and remaining time prediction of event sequences. In: Demeniconi C, Davidson I (eds) Proceedings of the 2021 SIAM International Conference on Data Mining, SDM 2021, Virtual Event, April 29–May 1, 2021, SIAM, Philadelphia, pp 522–530. <https://doi.org/10.1137/1.9781611976700.59>
- Taymouri F, Rosa ML, Erfani SM, Bozorgi ZD, Verenich I (2020) Predictive business process monitoring via generative adversarial nets: The case of next event prediction. In: Business Process Management - 18th Int. Conf., BPM 2020, Proc., Springer, Berlin-Heidelberg, LNCS, vol 12168, pp 237–256. https://doi.org/10.1007/978-3-030-58666-9_14

- Teinemaa I, Dumas M, La Rosa M, Maggi FM (2019) Outcome-oriented predictive process monitoring: Review and benchmark. *ACM Trans Knowl Discov Data* 13(2). <https://doi.org/10.1145/3301300>
- Tomar S, Gupta A (2023) A review on mode collapse reducing gans with gan's algorithm and theory. In: *GANs for Data Augmentation in Healthcare*, Springer International Publishing, Berlin-Heidelberg, pp 21–40. https://doi.org/10.1007/978-3-031-43205-7_2
- van der Aalst WMP (2016) *Process Mining - Data Science in Action*, Second Edition. Springer, Berlin-Heidelberg. <https://doi.org/10.1007/978-3-662-49851-4>
- van Dongen B (2012) Bpi challenge 2012. 4TU.ResearchData. Eindhoven University of Technology. <https://doi.org/10.4121/UUID:3926DB30-F712-4394-AEBC-75976070E91F>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.